



PROYECTO DE ORDINARIO

PORTAFOLIO DE EVIDENCIAS

PROGRAMACIÓN ESTRUCTURADA

Salinas Pérez Carolina Quetzali

**Ingeniería en
Física Aplicada**

217-A

**Universidad Tecnológica de la
Mixteca**





PRÓLOGO

El presente reporte tiene como finalidad documentar y analizar el proceso de aprendizaje consolidado durante el curso de Programación Estructurada, correspondiente al segundo semestre de la ingeniería en Física Aplicada. A lo largo de la asignatura, nos enfocamos en el dominio del lenguaje C pilar fundamental de la computación moderna utilizando el entorno de desarrollo Geany como herramienta principal para la compilación y depuración de algoritmos.

Además de la sintaxis, el objetivo central fue la interiorización del paradigma de programación estructurada. Este enfoque nos ha permitido desarrollar un pensamiento lógico-analítico para desglosar problemas complejos en subrutinas manejables, una habilidad crítica para cualquier ingeniero. Asimismo, se exploró el potencial de este lenguaje en la resolución de problemáticas dentro del ámbito tecnológico y científico, tales como la modelación de fenómenos físicos y el procesamiento de datos numéricos.

A continuación, se presentan los resultados obtenidos, las competencias técnicas desarrolladas y una serie de propuestas de mejora orientadas a optimizar la aplicación de estas herramientas en futuros proyectos de simulación y cálculo avanzado.



+ + INTRODUCCIÓN + +

El lenguaje de programación C se ha consolidado como una herramienta fundamental en la ingeniería debido a su excepcional rapidez y eficiencia. Desarrollado originalmente por Dennis Ritchie en los años 70 para el sistema operativo UNIX, su diseño evolucionó hacia la estandarización y portabilidad, permitiendo que el software desarrollado bajo este esquema sea capaz de ejecutarse en diversas arquitecturas con ajustes mínimos.

Esta versatilidad se debe, en gran medida, al proceso de compilación. A diferencia de otros lenguajes, C requiere de un traductor (como el compilador GCC) que transforma el código fuente en un archivo ejecutable, permitiendo así que el hardware comprenda y procese las instrucciones directamente.

Desde una perspectiva estructural, la programación en C se articula a través de funciones, siendo la función `main` el núcleo de ejecución de cualquier algoritmo. Para gestionar la información dentro de este flujo, el lenguaje emplea tipos de datos (enteros, flotantes y caracteres) y estructuras de control que determinan el comportamiento lógico del programa. Sin embargo, lo que realmente distingue a C es su capacidad de bajo nivel mediante el uso de punteros, los cuales permiten la manipulación directa de las direcciones de memoria, otorgando al programador un control total sobre los recursos del sistema.

Para facilitar el desarrollo, existen entornos de desarrollo integrado (IDE) como Geany. Esta herramienta es especialmente valiosa en el ámbito educativo, ya que unifica el editor de texto, el compilador y la consola en una sola interfaz. No obstante, el proceso técnico —ya sea en un IDE o mediante la terminal en sistemas como GNU/Linux— sigue un ciclo riguroso: la escritura del código, la depuración de errores sintácticos durante la compilación y, finalmente, la creación de un proceso gestionado por el sistema operativo que interactúa con el usuario mediante periféricos de entrada y salida.

En conclusión, la vigencia de C no es casualidad; su eficiencia y flexibilidad lo mantienen como el estándar indiscutible para el desarrollo de sistemas operativos, software embebido y aplicaciones de alto rendimiento, donde cada ciclo de reloj y cada byte de memoria son críticos.



EL LENGUAJE DE PROGRAMACIÓN ANSI C

C es un lenguaje de programación que permite interactuar de forma directa con el usuario, y, de manera indirecta, con el hardware del dispositivo. El lenguaje de programación ANSI C es una versión estandarizada del lenguaje C, definida por el organismo ANSI (American National Standards Institute) para asegurar que el código sea compatible entre diferentes sistemas y compiladores. Es un lenguaje estructurado y eficiente, muy usado en el desarrollo de sistemas operativos, programas de alto rendimiento y software embebido.

En ANSI C se utilizan variables para almacenar datos, funciones para organizar el programa y estructuras de control como if, for y while para controlar el flujo de ejecución. Todo programa comienza en la función main(), desde donde se ejecutan las instrucciones. También es fundamental el uso correcto de tipos de datos como int, float y char, así como el manejo de memoria mediante punteros, lo que permite un control más directo sobre el hardware.

Gracias a su estandarización, ANSI C facilita que un mismo programa pueda compilarse en distintas plataformas sin necesidad de muchos cambios, lo que lo hace muy importante en la programación profesional.



2.1

ESTRUCTURA DE UN PROGRAMA

Un programa en lenguaje C está formado por diferentes partes que permiten organizar el código y hacerlo funcional. Cada sección cumple una función específica, como incluir librerías, definir variables o ejecutar instrucciones. Esta estructura facilita la lectura, el mantenimiento y la correcta ejecución del programa.

Todo programa en C comienza a ejecutarse desde la función principal llamada `main()`, y dentro de ella se colocan las instrucciones que la computadora seguirá paso a paso.

```
/* * Nombre del archivo: estructura_completa.c
 * Descripción: Ejemplo detallado que integra todos los
elementos
 * fundamentales del lenguaje C.
```

/// 1. BIBLIOTECAS (Directivas del preprocesador)

```
#include <stdio.h> // Para entrada/salida (printf, scanf)
#include <string.h> // Para manejo de cadenas
```

// 2. MACROS Y CONSTANTES (Preprocesador)

```
#define PI 3.14159
#define MAX_NOMBRE 50
```

// 3. DEFINICIÓN DE TIPOS (typedef)

```
typedef unsigned char BYTE; // Crea un alias para un tipo de dato
```

// 4. ESTRUCTURAS (Agrupación de datos de distinto tipo)

```
struct Usuario {
int id;
char nombre[MAX_NOMBRE];
float saldo; };

```



// 5. UNIONES (Espacio de memoria compartido entre diferentes tipos)

```
union Datos {  
    int entero;  
    float decimal;  
    char letra; };
```

// 6. VARIABLES GLOBALES (Accesibles desde cualquier parte del programa)

```
int contadorGlobal = 0;
```

// 7. PROTOTIPOS DE FUNCIONES (Declarar funciones antes de usarlas)

```
void saludar(char nombre[]);  
float calcularArea(float radio);
```

// 8. FUNCIÓN PRINCIPAL (Punto de entrada)

```
int main() {
```

// 9. DEFINICIÓN DE VARIABLES LOCALES (Diferentes tipos)

```
int edad = 25; // Entero  
float estatura = 1.75; // Punto flotante  
double precision = 0.9999; // Doble precisión  
char inicial = 'A'; // Carácter  
int lista[5] = {1, 2, 3}; // Arreglo  
// Instancia de Estructurastruct  
Usuario usuario1 = {101, "Gemini", 1500.50};  
// Instancia de Unión  
union Datos miDato;  
miDato.entero = 10; // Nota: En uniones solo un miembro es válido a la vez
```



// 10. LÓGICA Y LLAMADO A FUNCIONES

```
printf("--- Información del Programa ---\n");
saludar(usuario1.nombre);
```

```
float area = calcularArea(5.0);
printf("El área del círculo es: %.2f\n", area);
printf("ID de usuario: %d\n", usuario1.id);
return 0; // Indica que el programa terminó con éxito
}
```

// 11. DEFINICIÓN DE FUNCIONES (El cuerpo del código)

```
// Función que no devuelve valor (void)
void saludar(char nombre[]) {
    printf("Hola, %s! Bienvenido al sistema.\n", nombre);
    contadorGlobal++; // Modificando la variable global
}

// Función que devuelve un valor (float)
float calcularArea(float radio) {
    return PI * (radio * radio);
}
```

Preprocesador (#): Instrucciones que se ejecutan antes de compilar.

Las bibliotecas traen herramientas externas y las macros reemplazan texto (como PI).

typedef: Muy útil para hacer el código más legible, en lugar de escribir unsigned char siempre, usas BYTE.

En la Estructura, todos los campos existen al mismo tiempo (el espacio es la suma de sus partes).

En la Unión, todos los campos comparten la misma dirección de memoria (el espacio es el del campo más grande).

Prototipos son como el "índice" de un libro. Le dicen al compilador que más adelante habrá una función llamada "...".

Variables Locales: Viven solo dentro de los corchetes {} donde fueron creadas (normalmente dentro de main).



APRENDIZAJE

En esta práctica aprendí cómo se organiza un programa básico en lenguaje C. Comprendí la función de las librerías, especialmente `stdio.h`, para poder utilizar instrucciones de entrada y salida.

También reforcé la declaración de variables y el uso de funciones como `printf` para mostrar mensajes y `scanf` para capturar datos desde el teclado. Además, entendí que la función `main()` es el punto donde inicia la ejecución del programa.

Este ejercicio me ayudó a familiarizarme con la estructura general y el flujo básico de un programa.

```
//primero se debe colocar las librerias
#include <stdio.h>
// se pueden colocar
// definiciones de constantes
// definiciones de prototipos de funciones

int main()
{
    int x;
    // la funcion printf seutiliza para mostrar mensajes
    printf("Hello world\t");
    printf("Introduce el valor de x:");
    scanf("%d",&x);
    return 1;
} // fin de la funcion main
```

```
Terminal
Hello world    Introduce el valor de x:5
-----
(program exited with code: 1)
Press return to continue
```




MEJORA

- Utilizar `return 0;` para indicar una ejecución correcta.
- Mostrar el valor ingresado por el usuario.
- Agregar saltos de línea para una mejor presentación.
- Corregir y organizar mejor los comentarios.
- Validar que el usuario ingrese un número adecuado.

```
#include <stdio.h>
```

```
int main() {
```

```
    int x;
```

```
    printf("Hello world\n");
```

```
    printf("Introduce el valor de x: ");
```

```
    scanf("%d", &x);
```

```
    printf("El valor ingresado es: %d\n", x);
```

```
    return 0;
```

```
}
```

```
Terminal
Hello world
Introduce el valor de x: 5
El valor ingresado es: 5

-----
(program exited with code: 0)
Press return to continue
```



APRENDIZAJE

En esta práctica aprendí a utilizar variables de tipo char, las cuales sirven para almacenar un solo carácter. También reforcé el uso de las funciones printf y scanf para mostrar mensajes y capturar datos desde el teclado.

Además, comprendí que los caracteres tienen una representación numérica en la tabla ASCII, por lo que es posible convertir una letra a su valor numérico utilizando (int).

Este ejercicio me ayudó a entender la relación entre caracteres y números dentro de la programación.

```
#include <stdio.h>
int main()
{
    char a;
    printf("Presiona una letra:");
    scanf("%c",&a);
    printf("\n\n\t\t Presionaste: %c,su ASCII ES:%d",a,(int)a);
    return 0;
}
```

```
Terminal
Presiona una letra:1

Presionaste: 1,su ASCII ES:49

-----
(program exited with code: 0)
```



MEJORA

- Mejorar la presentación de los mensajes en pantalla.
- Permitir ingresar más de un carácter o palabras completas.
- Validar que el usuario realmente introduzca una letra.
- Investigar más sobre la tabla ASCII y otros códigos de caracteres.
- Agregar comentarios más claros y organizados.

```
#include <stdio.h>

int main() {
    char a;

    printf("Presiona una letra: ");
    scanf("%c", &a);

    printf("\nPresionaste: %c\n", a);
    printf("Su codigo ASCII es: %d\n", (int)a);

    return 0;
}
```

```
Presiona una letra: I
Presionaste: I
Su codigo ASCII es: 73
-----
(program exited with code: 0)
```



TIPO DE DATOS

En el lenguaje C, los tipos de datos se utilizan para definir la naturaleza de la información que una variable puede almacenar, así como la cantidad de memoria que el sistema debe reservar para ella. La correcta elección del tipo de dato es fundamental para garantizar el uso eficiente de la memoria y el correcto funcionamiento del programa. Los tipos enteros se emplean para representar números sin parte decimal. Dependiendo de las necesidades del programa y de la arquitectura del sistema, pueden declararse en sus variantes con signo o sin signo, y con distintos tamaños de almacenamiento, como cortos (short), estándar (int) o largos (long). Los tipos de punto flotante, también llamados reales, permiten representar números con parte decimal. Estos tipos ofrecen diferentes niveles de precisión y rango de valores, siendo los más comunes float, double y long double. El tipo carácter se utiliza para almacenar un solo símbolo, como letras, dígitos o caracteres especiales. Internamente, los caracteres se representan mediante valores enteros asociados a un conjunto de codificación, como ASCII. A partir de estos tipos básicos, el lenguaje C proporciona modificadores que permiten ajustar el rango de valores y el consumo de memoria de las variables, lo cual resulta especialmente importante al desarrollar programas portables y eficientes.



APRENDIZAJE

En esta práctica aprendí a utilizar los tipos de datos básicos en C como int, float, double y char, los cuales sirven para almacenar distintos tipos de información (números enteros, decimales y caracteres). También reforcé el uso de printf para mostrar información en pantalla, entendiendo que cada tipo de dato necesita un formato específico para imprimirse correctamente.

```
#include <stdio.h>

int main() {

    // Declaración de variables con diferentes tipos de datos
    int edad = 17;           // entero
    float altura = 1.72;     // decimal (precisión simple)
    double pi = 3.1415926535; // decimal (más precisión)
    char inicial = 'S';      // carácter

    // Mostrar valores en pantalla
    printf("Ejemplo de tipos de datos en C\n\n");

    printf("Entero (int): %d\n", edad);
    printf("Decimal (float): %.2f\n", altura);
    printf("Decimal (double): %.10lf\n", pi);
    printf("Caracter (char): %c\n", inicial);

    return 0;
}
```

Ejemplo de tipos de datos en C

```
Entero (int): 17
Decimal (float): 1.72
Decimal (double): 3.1415926535
Caracter (char): S
```



MEJORA

- Usar nombres de variables más descriptivos para mayor claridad.
- Agregar entrada de datos con scanf para hacerlo más dinámico.
- Organizar la salida con mejor formato visual.
- Evitar usar demasiada precisión en double si no es necesaria.

```
#include <stdio.h>

int main() {

    int edad = 17;
    float altura = 1.72;
    double pi = 3.14159;
    char inicial = 'S';

    printf("==== TIPOS DE DATOS EN C ====\\n\\n");

    printf("Edad: %d\\n", edad);
    printf("Altura: %.2f\\n", altura);
    printf("Pi: %.5lf\\n", pi);
    printf("Inicial: %c\\n", inicial);

    return 0;
}
```

```
==== TIPOS DE DATOS EN C ====
```

```
Edad: 17
Altura: 1.72
Pi: 3.14159
Inicial: S
```



OPERADORES

La precedencia de operadores es utilizada por el lenguaje C para evaluar las expresiones en un orden correcto. La mayoría de los operadores se asocian de izquierda a derecha; sin embargo, el operador de asignación se asocia de derecha a izquierda.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar operadores relacionales (>) para comparar valores y determinar cuál número es mayor. También reforcé el uso de operadores de asignación (=) para actualizar el valor máximo encontrado.

```
if(a > max)
    max = a;
```

```
if(b > max)
    max = b;
```

Dados 3 números mostrar cual es el minimo

Ingrese el primer numero: 1

Ingrese el segundo: 2

Ingrese el tercer numero: 3

El maximo es: 3



MEJORA

- Inicializar correctamente la variable max.
- Mostrar mensajes más claros.
- Validar números negativos.
- Utilizar estructuras más organizadas.

```
max = a;  
  
if(b > max)  
{  
    max = b;  
}  
  
if(c > max)  
{  
    max = c;  
}
```

```
Dados 3 numeros mostrar cual es el maximo  
  
Ingrese el primer numero: 1  
Ingrese el segundo numero: 2  
Ingrese el tercer numero: 3  
  
El maximo es: 3
```




2.4

INTRUCCIONES DE ENTRADA Y SALIDA

Toda la entrada y salida de los programas se realiza mediante flujos de caracteres, los cuales suelen finalizar con un carácter de nueva línea o con un indicador de fin de archivo (EOF). Como se mencionó en una sección anterior, cuando un programa se ejecuta, el sistema operativo le asocia tres flujos de datos estándar para su comunicación: stdin, stdout y stderr. El sistema operativo permite que estos flujos puedan ser redirigidos hacia otros programas o dispositivos mediante conductos, también conocidos como tuberías (pipes).

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí diferentes formas de entrada y salida de datos en C. En lugar de utilizar únicamente `scanf()`, el programa recibe información desde la línea de comandos mediante los parámetros `argc` y `argv[]`. También aprendí a leer argumentos enviados al ejecutar el programa. Convertir cadenas de texto a números usando `atoi()`. Mostrar resultados en pantalla utilizando `printf()`. Validar si el usuario ingresó correctamente los datos.

```
n = atoi(argv[1]);  
  
printf("\n\nEl factorial de tu numero %d es %d\n\n", n, r);
```



MEJORA

- Validar mejor los argumentos ingresados.
- Mostrar instrucciones de uso.
- Evitar números negativos.
- Separar el cálculo factorial en una función.

```
if(argc != 2)
{
    printf("Debes ingresar un numero\n");
}
```



ESTRUCTURAS DE CONTROL

Las estructuras de control son el mecanismo fundamental que permite alterar el flujo de ejecución de un programa para que este deje de ser una simple sucesión lineal de instrucciones. En informática, su propósito es dotar al código de lógica y dinamismo, permitiendo que el procesador tome decisiones o repita procesos basándose en evaluaciones lógicas de datos. Estas estructuras funcionan como el sistema nervioso de un algoritmo; sin ellas, el software no podría reaccionar a diferentes entradas del usuario ni manejar excepciones, limitándose a ejecutar comandos de inicio a fin sin capacidad de respuesta. A nivel técnico, se basan en el uso de expresiones booleanas que determinan el camino que debe seguir el puntero de instrucción en la memoria. Su importancia radica en que permiten optimizar el rendimiento del hardware, garantizar la predictibilidad del software y, sobre todo, implementar soluciones complejas mediante la organización del código en bloques lógicos. En esencia, son la herramienta que transforma una secuencia estática de datos en un proceso inteligente capaz de resolver problemas, validar información y controlar el comportamiento del sistema de manera eficiente y estructurada

SECUENCIAL



Es la estructura de control más básica y el estado por defecto de cualquier programa. En este modelo, las instrucciones se ejecutan en una secuencia rígida, una tras otra, sin saltos ni desviaciones. Sigue una línea temporal donde la salida de una instrucción es, necesariamente, la entrada de la siguiente. No hay toma de decisiones; el programa simplemente "avanza"

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a desarrollar un programa secuencial, donde las instrucciones se ejecutan en orden, desde el inicio hasta el final, sin tomar decisiones ni repetir procesos.

También reforcé el uso de variables de tipo float para trabajar con números decimales y aprendí a capturar datos desde el teclado mediante scanf. Además, practiqué operaciones matemáticas básicas al calcular el área de un rectángulo multiplicando la base por la altura.

Este ejercicio me ayudó a comprender cómo resolver problemas matemáticos sencillos mediante programación.

```
#include <stdio.h>
int main()
{
    //declaracion de variables
    float a,b,h;
    printf("\t\t calcular el area de un rectangulo \n\n");
    printf("\t\t valor en unidades de la base:");
    // instruccion de lectura por teclado
    scanf("%f",&b);
    printf("\t\t valor en unidades de la altura:");
    // instruccion de lectura por teclado
    scanf("%f",&h);
    //calcular el area y asignarlo
    a=b*h;
    printf("El area es:%.2f",a);
    return 0;
} //fin del main
```

```
calcular el area de un rectangulo

valor en unidades de la base:9
valor en unidades de la altura:3

El area es:27.00

-----
(program exited with code: 0)
```



MEJORA

- Validar que la base y la altura sean valores positivos.
- Agregar mensajes más claros y ordenados.
- Utilizar nombres de variables más descriptivos.
- Mostrar unidades de medida en el resultado.
- Permitir calcular más áreas utilizando un menú o diferentes figuras.

```
#include <stdio.h>

int main() {
    float area, base, altura;

    printf("\t\tCALCULAR EL AREA DE UN RECTANGULO\n\n");
    printf("Ingresa el valor de la base: ");
    scanf("%f", &base);
    printf("Ingresa el valor de la altura: ");
    scanf("%f", &altura);
    area = base * altura;

    printf("\nEl area del rectangulo es: %.2f unidades cuadradas\n", area);

    return 0;
}
```

CALCULAR EL AREA DE UN RECTANGULO

```
Ingresa el valor de la base: 9
Ingresa el valor de la altura: 3
```

```
El area del rectangulo es: 27.00 unidades cuadradas
```



SELECTIVA

Estas estructuras permiten que el programa "elija" entre diferentes caminos basándose en una condición lógica. Es lo que permite que el software sea reactivo y tome decisiones según los datos que recibe. El programa llega a un punto de decisión donde evalúa una expresión (por ejemplo: ¿Es la edad mayor a 18?). Si el resultado es Verdadero, sigue un camino; si es Falso, sigue otro o simplemente ignora el bloque.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar estructuras selectivas (if y else) para tomar decisiones dependiendo de diferentes condiciones. También aprendí a generar números aleatorios con rand() para simular un juego del volado y a comparar la opción del usuario con el resultado obtenido.

```
vvolado = rand() % 2;

if (volado == 0) {

    if (x == 's') {
        printf("FELICIDADES... le atinaste\n");
    }

} else {

    if (x == 'a') {
        printf("FELICIDADES... le atinaste\n");
    }
}
}
```

---JUEGO DEL VOLADO---

Elige una opcion:

s) sol

a) aguilá

Opcion: a

cayo: sol

Que mal, no le atinaste



MEJORA

- Validar que el usuario ingrese únicamente las opciones correctas.
- Permitir jugar varias veces usando ciclos.
- Mejorar la presentación del programa.
- Agregar un contador de victorias y derrotas.

```
if (x != 's' && x != 'a') {  
    printf("Opcion invalida\n");  
}  
  
if ((volado == 0 && x == 's') ||  
    (volado == 1 && x == 'a')) {  
  
    printf("FELICIDADES... le atinaste\n");  
  
} else {  
  
    printf("Que mal, no le atinaste\n");  
}
```

```
Terminal  
  
--- JUEGO DEL VOLADO ---  
  
Elige una opcion:  
s) sol  
a) aguilá  
Opcion: a  
  
Cayo: aguilá  
FELICIDADES... le atinaste
```

APRENDIZAJE

En esta práctica aprendí a utilizar estructuras selectivas anidadas (if y else) para comparar varios números y ordenarlos de forma descendente. También reforcé el uso de operadores relacionales y lógicos para evaluar múltiples condiciones al mismo tiempo.

```
if (a >= b && a >= c) {  
  
    if (b >= c)  
        printf("%d, %d, %d\n", a, b, c);  
  
}  
else if (b >= a && b >= c) {  
  
    if (a >= c)  
        printf("%d, %d, %d\n", b, a, c);  
  
}  
else {  
  
    if (a >= b)  
        printf("%d, %d, %d\n", c, a, b);  
  
}
```

```
---PROGRAMA QUE DADO 3 NUMEROS LOS MUESTRA EN PANTALLA DE FORMA DESCENDENTE---  
  
Hola soy Carolina Q. Salinas P. del 217-A. ing Fisica Aplicada  
  
Introduce el primer numero: 5  
Introduce el segundo numero: 17  
Introduce el tercer numero: 22  
  
Los Numeros en orden descendente son: 22, 17, 5
```




MEJORA

- Corregir condiciones repetidas.
- Simplificar el código para hacerlo más corto y entendible.
- Validar correctamente los datos ingresados.
- Utilizar arreglos y ciclos para ordenar más números.

```
if (a < b) {  
    temp = a;  
    a = b;  
    b = temp;  
}
```

```
if (a < c) {  
    temp = a;  
    a = c;  
    c = temp;  
}
```

```
if (b < c) {  
    temp = b;  
    b = c;  
    c = temp;  
}
```

```
--- ORDENAR 3 NUMEROS DE FORMA DESCENDENTE ---
```

```
Introduce el primer numero: 5
```

```
Introduce el segundo numero: 17
```

```
Introduce el tercer numero: 22
```

```
Numero en orden descendente: 22, 17, 5
```



APRENDIZAJE

En esta práctica aprendí a utilizar estructuras selectivas (if) para comparar diferentes valores y encontrar el número mínimo entre tres datos ingresados por el usuario. También reforcé el uso de variables auxiliares para almacenar resultados temporales.

```
if(a < min)  
    min = a;
```

```
if(b < min)  
    min = b;
```

```
if(c < min)  
    min = c;
```

```
Dados 3 números mostrar cual es el minimo
```

```
Ingrese el primer numero:      22
```

```
Ingrese el segundo numero:     25
```

```
Ingrese el tercer numero:      28
```

```
El minimo es: 22
```



MEJORA

- Utilizar un valor inicial más adecuado para la variable min.
- Mejorar la presentación de los mensajes.
- Validar los datos ingresados.
- Hacer el programa compatible con más números utilizando arreglos y ciclos.

```
min = a;
```

```
if (b < min)  
    min = b;
```

```
if (c < min)  
    min = c;
```

```
--- ENCONTRAR EL NUMERO MINIMO ---
```

```
Ingrese el primer numero: 22  
Ingrese el segundo numero: 25  
Ingrese el tercer numero: 28
```

```
El numero minimo es: 22
```



APRENDIZAJE

En esta práctica aprendí a utilizar la estructura switch para seleccionar diferentes opciones dentro de un programa. También reforcé el uso de operaciones matemáticas para realizar conversiones de unidades y aprendí a organizar mejor un menú interactivo.

```
switch(elt) {  
  
    case 'A':  
        resultado = cantidad / 1000;  
        break;  
  
    case 'B':  
        resultado = cantidad * 35.274;  
        break;  
  
    case 'C':  
        resultado = cantidad / 16;  
        break;  
  
    default:  
        printf("Opcion no valida");  
}
```

Hola aqui te mostraré la Conversion básica de Unidades de Masa

(recuerda escribir en mayusculas)

Opción A : GRAMO (G) a KILOGRAMO (KG)

Opción B : KILOGRAMO(KG) a ONZA (OZ)

Opción C : ONZA (OZ) a LIBRA (LB)

Selecciona una opcion: A

Ingresa la cantidad a convertir: 55

55.00 Gramos son 0.0550 Kilogramos



MEJORA

- Validar que las cantidades sean positivas.
- Permitir ingresar opciones en minúsculas y mayúsculas.
- Agregar más conversiones de unidades.
- Mejorar la presentación del menú.

```
switch(elc) {  
  
    case 'A':  
    case 'a':  
        resultado = cantidad / 1000;  
        break;  
  
    case 'B':  
    case 'b':  
        resultado = cantidad * 35.274;  
        break;  
  
    case 'C':  
    case 'c':  
        resultado = cantidad / 16;  
        break;  
  
}
```

--- CONVERSION DE UNIDADES DE MASA ---

A) Gramos a Kilogramos
B) Kilogramos a Onzas
C) Onzas a Libras

Selecciona una opcion: A
Ingresa la cantidad: 55

55.00 g = 0.0550 kg



ITERATIVAS

Son estructuras diseñadas para repetir un bloque de instrucciones múltiples veces. Son la herramienta de potencia de la programación, ya que permiten procesar miles de datos con apenas unas líneas de código. El programa repite el código mientras una condición de permanencia sea verdadera. En el momento en que la condición se vuelve falsa, el ciclo se rompe y el flujo continúa con la siguiente parte del programa.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar estructuras repetitivas (for) para realizar sumatorias automáticamente. También aprendí a utilizar funciones matemáticas como `pow()` para calcular potencias y resolver operaciones más complejas dentro de un ciclo.

```
for (n=1; n<=x; n++)  
{  
    termino =(1.0/ pow(2,n));  
    s=s+termino;  
}
```

```
-----SUMA DE ELEMENTOS de las suceción an=1/2^n -----  
  
Limite superior de la suma:      55  
  
    Tu sumatoria es de =1.00000  
  
-----  
(program exited with code: 0)  
Press return to continue
```



MEJORA

- Validar mejor los datos ingresados.
- Mostrar cada término de la sucesión.
- Utilizar nombres de variables más descriptivos.
- Mejorar la presentación de resultados.

```
for (n = 1; n <= x; n++) {

    termino = 1.0 / pow(2, n);
    suma += termino;

    printf("Termino %d = %.5lf\n", n, termino);
}
```

```
--- SUMA DE LA SUCESION  $a_n = 1 / 2^n$  ---
```

```
Limite superior de la suma: 55
```

```
Termino 1 = 0.50000
```

```
Termino 2 = 0.25000
```

```
Termino 3 = 0.12500
```

```
Termino 4 = 0.06250
```

```
Termino 5 = 0.03125
```

```
Termino 6 = 0.01562
```

```
Termino 53 = 0.00000
```

```
Termino 54 = 0.00000
```

```
Termino 55 = 0.00000
```

```
La sumatoria es = 1.00000
```

APRENDIZAJE

En esta práctica aprendí a utilizar ciclos for para trabajar con intervalos y realizar cálculos repetitivos automáticamente. También reforcé el uso de operaciones matemáticas y fórmulas físicas, en este caso la Ley de Hooke, para calcular la fuerza y el estiramiento de un resorte según diferentes masas.

```
for (m = m_inicial; m < m_final; m += incremento)
{
    float fuerza = m * gravedad;
    float x = fuerza / k;

    printf("\nPara una masa de %.2f la fuerza es de %.2f
y el estiramiento es de %.2f",
        m, fuerza, x);
}
```

```
¡El presente programa te muestra la ley de hooke! :)

Ingresa un valor de tu constante K:25

¡Define el intervalo de masas a utilizar!

Dame tu masa minina: 12

Dame tu masa máxima: 52

Incremento de masa: 3

Los resultados obtenidos fueron:
```

```
Para una masa de 12.00 la fuerza es de 117.72 y el estiramiento es de 1.18
```

```
Para una masa de 48.00 la fuerza es de 470.88 y el estiramiento es de 18.84
```

```
Para una masa de 51.00 la fuerza es de 500.31 y el estiramiento es de 20.01
```

```
¡GRACIASSSS. Eso fue todo!
```

```
.....
(program exited with code: 0)
Press return to continue
```




MEJORA

- Validar que los datos ingresados sean positivos.
- Evitar divisiones entre cero cuando $k = 0$.
- Mejorar la presentación de resultados mediante tablas.
- Utilizar funciones para organizar mejor el programa.

```
if (k <= 0 || incremento <= 0) {  
    printf("Datos invalidos\n");  
}  
else {  
  
    for (m = m_inicial; m <= m_final; m += incremento) {  
  
        fuerza = m * gravedad;  
        x = fuerza / k;  
  
        printf("\nMasa: %.2f | Fuerza: %.2f | Estiramiento: %.2f",  
            m, fuerza, x);  
    }  
}
```

--- LEY DE HOOKE ---

Ingresa el valor de la constante k: 25

Masa minima: 12

Masa maxima: 52

Incremento de masa: 3

Resultados:

Masa: 12.00 | Fuerza: 117.72 | Estiramiento: 4.71

Masa: 15.00 | Fuerza: 147.15 | Estiramiento: 5.89

Masa: 18.00 | Fuerza: 176.58 | Estiramiento: 7.06



FUNCIONES



Las funciones representan el pilar de la programación modular, definiéndose como bloques de código independientes y reutilizables diseñados para ejecutar tareas específicas. La arquitectura de C es puramente procedimental, lo que significa que el flujo del programa se organiza a través de llamadas a estas unidades lógicas, siendo la función main el punto de entrada obligatorio de cualquier aplicación. Técnicamente, una función en C se compone de un tipo de retorno (que define el dato que devuelve, como int o float, o void si no retorna nada), un identificador único, una lista de parámetros opcionales y un cuerpo delimitado por llaves donde reside la lógica. El uso de funciones permite la abstracción de procesos, facilitando que el programador se enfoque en el "qué hace" un módulo sin necesidad de gestionar el "cómo lo hace" en cada invocación. Además, las funciones gestionan su propio ámbito de memoria, creando variables locales que solo existen durante su ejecución, lo que optimiza el uso de la RAM y evita conflictos de datos entre distintas partes del programa. Esta estructura no solo reduce la duplicidad de código, sino que mejora drásticamente la legibilidad, facilita el mantenimiento y permite la creación de bibliotecas que pueden ser integradas en múltiples proyectos, estableciendo así un entorno de desarrollo eficiente y organizado. Las funciones son un recurso que tiene el lenguaje C para poder hacer modular los programas, permitiendo al programador reutilizar códigos para ahorrar tiempo en el desarrollo de software. En este capítulo se verán las características que brinda el lenguaje para crear funciones, así como los tipos de pasos de parámetros que pueden ser usados para comunicarse con ellas.



APRENDIZAJE

En esta práctica aprendí a utilizar funciones para dividir un programa grande en partes más organizadas y fáciles de entender. También comprendí cómo enviar arreglos y estructuras como parámetros a diferentes funciones para registrar y buscar información de pacientes.

```
void altas(DATOS p[], int x);  
void mostrar(DATOS p[], int x);  
  
switch(opcion)  
{  
    case 1:  
        altas(paciente, contador);  
        contador++;  
        break;  
  
    case 2:  
        mostrar(paciente, contador);  
        break;  
}
```

```
=====
EXPEDIENTE      CLINICA CLINATOLOGICA
=====
Ingrese folio del paciente 🗨️ : 123
--- PACIENTE ENCONTRADO ---
Folio: 123
Nombre: Adrian Hernandez Salazar
Sexo: m
Edad: 18
Tipo de sangre: o+
Presion: 34/43
Temperatura: 27.00
Talla: 0.00
Presiona ENTER para continuar...|
```



MEJORA

- Organizar mejor las funciones del programa.
- Validar los datos ingresados.
- Agregar más funciones como editar o eliminar pacientes.
- Guardar la información en archivos para conservar los registros.

```

void menu();
void altas(DATOS p[], int x);
void mostrar(DATOS p[], int x);

do {
    menu();
    scanf("%d", &opcion);

    switch(opcion)
    {
        case 1:
            altas(paciente, contador);
            contador++;
            break;
        case 2:
            mostrar(paciente, contador);
            break;
    }
} while(opcion != 3);

```

```

===== CLINICA =====

1) Registrar paciente
2) Buscar paciente
3) Salir

Opcion: 2

Ingresa el folio: 123

===== CLINICA =====

1) Registrar paciente
2) Buscar paciente
3) Salir

Opcion: 1

Folio: 123
Nombre: adrian hernandez salazar
Edad: 18

===== CLINICA =====

```

DEFINICIÓN DE FUNCIONES

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar funciones para dividir un programa en partes más organizadas y reutilizables. También reforcé el uso de arreglos de cadenas, ciclos y estructuras selectivas para desarrollar un juego interactivo tipo “Ahorcado”.

```
while((vidas != 0) && (ganar != 1))
{
    for (i = 0; i < strlen(materia[x]); i++)
    {
        if (letra == materia[x][i])
        {
            aux[i] = letra;
            flag = 1;
        }
    }
}
```





MEJORA

- Evitar funciones no compatibles como `_fpurge`.
- Validar mejor las letras ingresadas.
- Agregar más palabras o categorías.
- Hacer el juego más modular utilizando más funciones.

```
if(strcmp(materia[x], aux) == 0)
{
    ganar = 1;
    printf("\nGanaste!!!\n");
}
```

```
=====
JUEGO DEL AHORCADO
=====

Palabra: *****
Vidas: 3
Ingresa una letra: g
Letra incorrecta

Palabra: *****
Vidas: 2
Ingresa una letra: s
Letra incorrecta
```

```
Palabra: *****
Vidas: 1
Ingresa una letra: y
Letra incorrecta
```

```
Perdiste
La palabra era: calculo
```

```
-----
(program exited with code: 0)
Press return to continue
```



4.2

FUNCIONES CON PASO DE PARAMETRO POR VALOR

Al pasar los parámetros por valor lo que se realiza es una copia del valor del argumento y las modificaciones a la copia no afectan al valor original de la variable. Se debe recordar que las variables declaradas en cada función son variables locales, es decir, solo son conocidas en dicha función.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a crear funciones que reciben parámetros por valor para realizar operaciones matemáticas. También reforcé el uso de switch para seleccionar distintas operaciones dependiendo de la opción elegida por el usuario.

```
c = suma(a,b);
```

```
c = multiplicacion(a,b);
```

```
Dame el primer numero:
12
Dame el segundo numero:
22
Presiona s si quieres saber la suma o m si quieres saber la multiplicacion:
m

Eligió ver la multiplicacion

La multiplicacion es: 264.00
```




MEJORA

- Agregar más operaciones.
- Validar opciones inválidas.
- Crear un menú más organizado.
- Evitar repetir código.

```
double suma(double x, double y)
{
    return x + y;
}

double multiplicacion(double x, double y)
{
    return x * y;
}
```

```
Dame el primer numero:
12
Dame el segundo numero:
22

Presiona:
s -> suma
m -> multiplicacion
Opcion: m

La multiplicacion es: 264.00
```


FUNCIONES POR PASO DE PARÁMETROS POR REFERENCIA

```
void suma(double x, double y, double *r)
```

En el lenguaje C, el paso por referencia se logra técnicamente a través de apuntadores. En lugar de enviarle a la función una copia del dato (como un "5" o un "10"), se le envía la dirección de memoria donde está guardada la variable original.

PRACTICAS

1

```
void multiplicacion(double x, double y, double *r)
```

```
{
    *r = x * y;
}
```

APRENDIZAJE

En esta práctica aprendí a utilizar funciones con paso de parámetros por referencia en C, lo que permite que una función modifique directamente el valor de una variable externa utilizando punteros (*) y la dirección de memoria (&).

También reforcé el uso de funciones con parámetros, entendiendo que en lugar de regresar un valor con return, se puede modificar una variable directamente desde la función.

```
void suma(double x, double y, double *r)
{
    *r = x + y;
}

void multiplicacion(double x, double y, double *r)
{
    *r = x * y;
}

suma(a, b, &c);
multiplicacion(a, b, &c);
```

```
Dame el primer numero:
13
Dame el segundo numero:
23
Presiona s si quieres la suma o m si quieres la multiplicacion:
s

Eligio ver la suma

La suma es: 36.00
```



MEJORA

- Evitar `_fpurge(stdin)` porque no es estándar en todos los compiladores
- Validar mejor la entrada del usuario (aceptar mayúsculas y minúsculas)
- Agregar más operaciones (resta, división, potencia)
- Mejorar mensajes para que sean más claros y uniformes
- Separar entrada, proceso y salida para mejor organización del código

```
void suma(double x, double y, double *r);  
void multiplicacion(double x, double y, double *r);  
  
suma(a, b, &c);  
multiplicacion(a, b, &c);
```

```
Dame el primer numero:  
13  
Dame el segundo numero:  
23  
Elige operacion (s = suma, m = multiplicacion):  
s  
Resultado de la suma: 36.00
```

FUNCIONES RECURSIVAS



Se denomina función recursiva a aquella que se invoca a sí misma para resolver un problema, descomponiéndolo en versiones más simples del mismo. Una característica fundamental de toda función recursiva es la existencia de un caso base, el cual establece de manera explícita la condición bajo la cual la función deja de llamarse a sí misma.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar funciones recursivas para resolver problemas matemáticos, en este caso convertir un número decimal a binario. También reforcé el uso de parámetros y llamadas a funciones dentro del programa.

```
f (n > 0) {  
  
    binario(n / 2);  
  
    printf("%d", n % 2);  
}
```

```
Ingresa el valor que decaas conocer en binario :122
```

```
Tu numero en binario es:1111010
```

```
-----  
(program exited with code: 0)  
Press return to continue
```



MEJORA

- Validar que el número ingresado sea positivo.
- Mejorar los mensajes del programa.
- Separar mejor la lógica del programa.
- Explicar con comentarios cómo funciona la recursividad.

```
void binario(int n)
{
    if(n > 1)
        binario(n / 2);
    printf("%d", n % 2);
}
```

```
Ingresa un numero decimal: 122
```

```
Numero en binario: 1111010
```

```
-----
(program exited with code: 0)
Press return to continue
```



ARREGLOS

Un arreglo en el lenguaje C es una estructura de datos fundamental que permite agrupar una colección finita, homogénea y estática de elementos bajo un mismo nombre. Su naturaleza homogénea implica que todos los datos almacenados deben ser del mismo tipo, mientras que su carácter estático obliga a definir un tamaño fijo desde el momento de su creación, el cual no puede alterarse durante la ejecución del programa. Técnicamente, los arreglos se caracterizan por la contigüidad en memoria, lo que significa que el sistema reserva un bloque de direcciones físicas consecutivas para almacenar los elementos; esta disposición permite que el acceso a cualquier dato sea extremadamente rápido mediante el uso de índices. En C, el indexado es de base cero, por lo que el primer elemento se ubica siempre en la posición 0 y el último en la posición N-1. Dependiendo de su organización, los arreglos pueden ser unidimensionales, conocidos como vectores, o multidimensionales, como las matrices que organizan la información en filas y columnas. El uso de arreglos es esencial para la gestión eficiente de grandes volúmenes de información, permitiendo automatizar el procesamiento de datos a través de estructuras iterativas y sirviendo como base para la construcción de estructuras de datos más avanzadas y complejas.



5.1

ARREGLOS UNIDIMENSIONALES

Los arreglos unidimensionales, comúnmente conocidos como vectores o listas, son la forma más sencilla de agrupación de datos en C. Se definen como una lista lineal de elementos del mismo tipo que comparten un único nombre y se distinguen entre sí por un solo índice de posición.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a trabajar con arreglos unidimensionales para almacenar y combinar conjuntos de datos. También reforcé la búsqueda de elementos dentro de arreglos para evitar repetir valores al formar un nuevo arreglo.

```
for (i = 0; i < 10; i++)
{
    flag = 0;

    for (j = 0; j < 10; j++)
    {
        if(B[i] == A[j])
        {
            flag = 1;
        }
    }

    if(flag == 0)
    {
        C[h] = B[i];
        h++;
    }
}
```

Valores del conjunto C:

```
C[0]=1
C[1]=2
C[2]=3
C[3]=4
C[4]=5
C[5]=6
C[6]=7
C[7]=8
C[8]=9
C[9]=10
C[10]=12
C[11]=14
C[12]=16
C[13]=18
C[14]=20
```



MEJORA

- Usar nombres de variables más descriptivos.
- Mostrar mejor los resultados.
- Evitar valores repetidos utilizando funciones.
- Permitir que el usuario ingrese los valores.

```
for(i = 0; i < 10; i++)
{
    repetido = 0;

    for(j = 0; j < h; j++)
    {
        if(B[i] == C[j])
        {
            repetido = 1;
        }
    }

    if(!repetido)
    {
        C[h++] = B[i];
    }
}
```

Valores del conjunto C:

Elementos del arreglo C:

```
C[0] = 1
C[1] = 2
C[2] = 3
C[3] = 4
C[4] = 5
C[5] = 6
C[6] = 7
C[7] = 8
C[8] = 9
C[9] = 10
C[10] = 12
C[11] = 14
C[12] = 16
C[13] = 18
C[14] = 20
```

APRENDIZAJE

En esta práctica aprendí a recorrer cadenas de caracteres utilizando arreglos unidimensionales y a evaluar caracteres mediante condiciones para identificar vocales y consonantes.

```
for(i = 0; i < strlen(palabra); i++)  
{  
    if((palabra[i] != 'a') &&  
        (palabra[i] != 'e') &&  
        (palabra[i] != 'i') &&  
        (palabra[i] != 'o') &&  
        (palabra[i] != 'u'))  
  
        v++;  
}
```

Dame la palabra: ordinario

Numero de consonantes: 4



MEJORA

- Contar vocales y consonantes por separado.
- Aceptar mayúsculas.
- Permitir palabras más largas.
- Validar caracteres especiales.

```
for(i = 0; i < strlen(palabra); i++)  
{  
    if(palabra[i]=='a' || palabra[i]=='e'  
    ||  
        palabra[i]=='i' || palabra[i]=='o'  
    ||  
        palabra[i]=='u')  
    {  
        vocales++;  
    }  
    else  
    {  
        consonantes++;  
    }  
}
```

Dame una palabra: ordinario

Numero de vocales: 5

Numero de consonantes: 4



5.2

ARREGLOS BIDIMENSIONALES

Los arreglos bidimensionales, conocidos técnicamente como matrices, son estructuras que organizan los datos en un formato de tabla, compuesto por filas y columnas. Es la evolución lógica de un vector: si un arreglo unidimensional es una línea, el bidimensional es un plano.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar arreglos bidimensionales para almacenar varias cadenas de texto dentro de una sola variable. También reforcé el uso de funciones y números aleatorios para seleccionar palabras al azar.

```
char palabras[7][16] =  
{  
    {"jirafa"},  
    {"hipopotamo"},  
    {"tuza"},  
    {"mapache"}  
};
```

```
palabra: rroncorrorrinco
```





MEJORA

- Mostrar mensajes más organizados.
- Validar el tamaño de las palabras.
- Separar el dibujo en más funciones.
- Permitir que el usuario adivine letras.

```
x = rand() % 7;
```

```
printf("\nPalabra seleccionada: %s\n", palabras[x]);
```

```
Palabra seleccionada: pinguino
```



APRENDIZAJE

En esta práctica aprendí a trabajar con matrices utilizando arreglos bidimensionales. También reforcé el uso de ciclos anidados (for) para recorrer filas y columnas y obtener la matriz transpuesta.

```
for(i = 0; i < 3; i++)  
for(j = 0; j < 3; j++)  
{  
    scanf("%f", &A[i][j]);  
}
```

Valores de la matriz A

```
A[0][0]= 1 2  
A[0][1]= A[0][2]= 3 4  
A[1][0]= A[1][1]= 5 6  
A[1][2]= A[2][0]= 7 8  
A[2][1]= A[2][2]= 9 10  
A[0][0]= 1.0  
A[1][0]= 4.0  
A[2][0]= 7.0  
A[0][1]= 4.0  
A[1][1]= 5.0  
A[2][1]= 8.0  
A[0][2]= 7.0  
A[1][2]= 8.0  
A[2][2]= 9.0
```



MEJORA

- Crear una matriz aparte para la transpuesta.
- Mostrar la matriz en formato ordenado.
- Validar los datos ingresados.
- Evitar modificar directamente la matriz original.

```
for(i = 0; i < 3; i++)  
{  
    for(j = 0; j < 3; j++)  
    {  
        T[i][j] = A[j][i];  
    }  
}
```

```
Valores de la matriz A  
A[0][0] = 1 2  
A[0][1] = A[0][2] = 3 4  
A[1][0] = A[1][1] = 5 6  
A[1][2] = A[2][0] = 7 8  
A[2][1] = A[2][2] = 9 10
```

```
Matriz transpuesta:  
1.0    4.0    7.0  
2.0    5.0    8.0  
3.0    6.0    9.0
```



5.3

ARREGLOS COMO PARÁMETROS

En el lenguaje C, los arreglos se pasan a las funciones exclusivamente por referencia, enviando la dirección de memoria de su primer elemento en lugar de una copia completa de los datos.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar arreglos como parámetros en funciones en C, lo cual permite enviar un conjunto de datos a una función para que sean procesados de manera conjunta. Entendí que un arreglo no es una sola variable, sino un conjunto de valores que se recorren mediante un ciclo for. También aprendí que los arreglos pueden pasarse a las funciones sin necesidad de copiar cada elemento, ya que la función trabaja directamente con los datos del arreglo original.

```
int sumaArreglo(int arreglo[], int tamaño)
{
    int suma = 0;

    for(int i = 0; i < tamaño; i++)
    {
        suma = suma + arreglo[i];
    }

    return suma;
}
```

```
Ingresa 5 numeros:
Numero 1: 10
Numero 2: 22
Numero 3: 17
Numero 4: 12
Numero 5: 18

La suma de los elementos del arreglo es: 79
```



MEJORA

- Permitir que el usuario decida el tamaño del arreglo
- Validar entradas para evitar errores (solo números válidos)
- Separar la lógica en más funciones (llenado y suma)
- Usar constantes (#define) para el tamaño del arreglo
- Mejorar mensajes para mayor claridad

```
int sumaArreglo(int arreglo[], int tamaño);  
  
resultado = sumaArreglo(numeros, TAM);
```

```
Ingresa 5 numeros:
```

```
Numero 1: 10
```

```
Numero 2: 22
```

```
Numero 3: 17
```

```
Numero 4: 12
```

```
Numero 5: 18
```

```
La suma de los elementos del arreglo es: 79
```



5.4

ORDENAMIENTO Y BÚSQUEDAS EN AREGLOS

El ordenamiento y la búsqueda son los dos procesos más comunes aplicados a los arreglos. El primero organiza los datos para que tengan sentido (numérico o alfabético), y el segundo permite localizar un dato específico en el menor tiempo posible.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a ordenar y buscar datos en un arreglo en C. Primero utilicé el método de burbuja para ordenar los números de menor a mayor, comparando e intercambiando elementos dentro del arreglo. Después realicé una búsqueda lineal, recorriendo el arreglo para encontrar un número específico ingresado por el usuario.

```
ordenar(numeros, 5);  
pos = buscar(numeros, 5, valor);
```

Ingresa 5 numeros:

18
12
17
22
10

Arreglo ordenado:

10 12 17 18 22



MEJORA

- Optimizar el ordenamiento
- Mejorar mensajes de salida
- Permitir tamaño dinámico del arreglo
- Separar el llenado en una función

```
for(int j = 0; j < n - 1 - i; j++)  
    if(arreglo[j] > arreglo[j + 1])
```

Ingresa 5 numeros:

18

12

17

22

10

Arreglo ordenado:

10 12 17 18 22



APUNTADORES



Un apuntador es una variable especial en el lenguaje C cuyo valor no es un dato común (como un número o un carácter), sino la dirección de memoria de otra variable. Se consideran una de las herramientas más potentes del lenguaje, ya que permiten la manipulación directa de la memoria RAM y la gestión dinámica de recursos. Para su funcionamiento, se utilizan dos operadores clave: el operador de dirección `&`, que extrae la ubicación de una variable, y el operador de desreferenciación `*`, que permite acceder o modificar el contenido almacenado en esa dirección. Los apuntadores son fundamentales para implementar el paso de parámetros por referencia, permitiendo que las funciones modifiquen variables externas, y son la base para el manejo de arreglos, cadenas de texto y la creación de estructuras de datos complejas como listas ligadas y árboles. Su uso correcto optimiza drásticamente el rendimiento del software, aunque requiere una gestión precisa para evitar errores de acceso a memoria no autorizada.



6.1

APUNTADORES

Un apuntador es una variable que almacena la dirección de memoria de otra variable en lugar de un valor común. Es la herramienta que permite manipular la memoria RAM de forma directa y eficiente.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar ciclos for para calcular el factorial de un número. También aprendí a trabajar con argumentos enviados desde la línea de comandos mediante argc y argv, así como a convertir cadenas a enteros usando atoi().

```
n = atoi(argv[1]);  
  
for(int i = 1; i <= n; i++)  
    r = r * i;
```

```
Terminal  
Error: Debes ingresar un número como argumento.  
-----  
(program exited with code: 1)  
Press return to continue
```



MEJORA

- Validar mejor los argumentos ingresados.
- Evitar desbordamientos con números grandes.
- Mostrar instrucciones al usuario cuando no ingrese datos.
- Utilizar funciones para organizar el programa.

```
if (argc != 2) {  
    printf("Debes ingresar un numero\n");  
}  
else {  
  
    n = atoi(argv[1]);  
  
    for(int i = 1; i <= n; i++) {  
        r *= i;  
    }  
}
```

```
Debes ingresar un numero  
  
-----  
(program exited with code: 1)  
Press return to continue  
█
```



6.2

ARITMÉTICA DE APUNTADORES

La aritmética de apuntadores es una característica única de C que permite realizar operaciones matemáticas sobre las direcciones de memoria. A diferencia de la aritmética común, aquí los cálculos no se basan en unidades decimales, sino en el tamaño del tipo de dato al que apunta el puntero.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a utilizar apuntadores para recorrer y manipular cadenas de texto. También aprendí a trabajar con memoria dinámica usando malloc() y a comparar caracteres para identificar si una frase es un palíndromo.

```
while(*p != '\0')
{
    if(*p != ' ' && *p != '\n')
    {
        *q = *p;
        q++;
        cont++;
    }
    p++;
}
```

```
Terminal
Ingresa una frase: Pongame 10 profe, please!!!
Tu frase sin espacios: Pongame10profe,please!!!
Tu frase invertida: !!!esaelp,eforp01emagnoP
Tu frase no es palindromo
```



MEJORA

- Liberar la memoria usando free().
- Validar correctamente la memoria reservada.
- Ignorar mayúsculas y minúsculas al comparar.
- Hacer el código más organizado utilizando funciones.

```
while(*p != '\0')
{
    if(*p != ' ' && *p != '\n')
    {
        *q = tolower(*p);
        q++;
        cont++;
    }
    p++;
}
```

```
Ingresa una frase: Pongame 10 profe, please!!!
Frase sin espacios: pongame10profe,please!!!
Frase invertida: !!!esaelp,eforpo1emagnop
La frase no es palindromo
```



6.3

MEMORIA ESTÁTICA Y DINÁMICA

La gestión de memoria en C se divide en dos tipos fundamentales: estática y dinámica. La memoria estática se reserva en tiempo de compilación y se almacena en una estructura llamada Stack (pila); su tamaño es fijo, no puede alterarse durante la ejecución y el sistema la libera automáticamente cuando la función o el programa terminan. Por el contrario, la memoria dinámica se solicita durante el tiempo de ejecución y se ubica en el Heap (montículo), permitiendo al programador definir y modificar el tamaño de las estructuras de datos según las necesidades del momento. Para gestionar la memoria dinámica, se utilizan funciones específicas como `malloc()` o `calloc()` para reservar espacio y, de manera obligatoria, la función `free()` para liberar dicho espacio manualmente. Mientras que la memoria estática es más rápida y segura ante fugas, la memoria dinámica ofrece la flexibilidad necesaria para manejar grandes volúmenes de datos o estructuras de tamaño variable, siendo responsabilidad total del programador evitar el desperdicio de recursos o errores de acceso.



TIPO DE DATOS DEFINIDOS POR EL PROGRAMADOR

En el lenguaje C, los tipos de datos definidos por el programador permiten agrupar diversas variables bajo un mismo nombre para representar entidades complejas de la vida real, como un "Alumno", un "Producto" o un "Vehículo". La herramienta principal para esto es la estructura (struct), que actúa como una plantilla para almacenar múltiples datos de distintos tipos en una sola unidad lógica. Complementariamente, el uso de la palabra reservada typedef es fundamental, ya que permite crear alias o sobrenombres para tipos de datos existentes, incluyendo las propias estructuras. Al emplear typedef, se simplifica drásticamente la sintaxis del código, eliminando la necesidad de repetir la palabra struct en cada declaración y mejorando la legibilidad y el mantenimiento del software. Esta capacidad de personalización transforma a C en un lenguaje versátil, permitiendo que el programador diseñe abstracciones de datos a medida que se adaptan perfectamente a la lógica del problema que se intenta resolver.

SIN EL USO DE TYPEDEF

```
struct Estudiante {  
  
    int id;  
    char nombre[50];  
    float promedio;  
  
}; // Declaración de variable  
  
struct Estudiante alumno1;
```

CON EL USO DE TYPEDEF

```
typedef struct {  
    int id;  
    char nombre[50];  
    float promedio;  
} Estudiante;  
  
// Declaración de variable  
// (mucho más limpia)  
Estudiante alumno1;
```




ARCHIVOS

El manejo de archivos en C es lo que permite la persistencia de datos, es decir, que la información no se pierda al cerrar el programa y pueda ser almacenada permanentemente en el disco duro.

El manejo de archivos en el lenguaje C se realiza a través de un tipo de dato especial llamado flujo o stream, representado por el puntero de tipo FILE. Para interactuar con un archivo, el programa debe seguir un ciclo de tres pasos obligatorios: apertura, procesamiento y cierre. La apertura se realiza mediante la función `fopen()`, la cual requiere el nombre del archivo y un modo de acceso (como "r" para lectura, "w" para escritura o "a" para añadir datos). Una vez abierto, se pueden utilizar funciones como `fprintf()` y `fscanf()` para archivos de texto, o `fwrite()` y `fread()` para archivos binarios, permitiendo la manipulación de grandes volúmenes de información de manera organizada. Finalmente, es imperativo utilizar la función `fclose()` para liberar los recursos del sistema y asegurar que todos los datos pendientes se escriban correctamente en el disco. La gestión de archivos es la base para la creación de bases de datos, registros de actividad (logs) y la configuración de aplicaciones, permitiendo que la información trascienda la ejecución volátil de la memoria RAM.

**APRENDIZAJE**

En esta práctica aprendí a crear y manipular archivos utilizando `fopen()`, `fprintf()` y `fclose()`. También aprendí a guardar información ingresada por el usuario dentro de un archivo de texto y a utilizar `strcmp()` para finalizar el programa cuando se escriba una palabra específica.

```
archivo = fopen("historia.dat", "a");  
  
fprintf(archivo, "%s", linea);  
  
fclose(archivo);
```

```
Escribe texto (escribe 'fin' para terminar):  
holaaaaa, xd  
fin  
  
Datos guardados correctamente  
  
-----  
(program exited with code: 0)  
Press return to continue
```



MEJORA

- Validar mejor la apertura del archivo.
- Mostrar mensajes de confirmación al guardar.
- Permitir leer el contenido del archivo.
- Evitar funciones innecesarias como `stdio_ext.h`.

```
if(strcmp(linea, "fin\n") == 0)
    break;

fprintf(archivo, "%s", linea);
```

```
Escribe texto (escribe 'fin' para terminar):
holaaaaa, xd
fin

Datos guardados correctamente

-----
(program exited with code: 0)
Press return to continue
```



TEXTO Y BINARIOS

Son bloques de código independientes y reutilizables. Se definen con un tipo de retorno, nombre y parámetros. El paso por valor envía una copia del dato (protege el original), mientras que el paso por referencia envía la dirección de memoria mediante apuntadores, permitiendo modificar la variable original. La recursividad es cuando una función se invoca a sí misma, requiriendo siempre un caso base para evitar bucles infinitos.

PRACTICAS

1

APRENDIZAJE

En esta práctica aprendí a trabajar con archivos binarios utilizando `fwrite()` para guardar diferentes tipos de datos dentro de un archivo. También reforcé el uso de `fopen()` y `fclose()` para crear y manipular archivos en modo binario.

```
archivo = fopen("datos.bin","wb");
```

```
fwrite(&a, sizeof(a), 1, archivo);
```

```
fwrite(&b, sizeof(b), 1, archivo);
```

```
fwrite(&c, sizeof(c), 1, archivo);
```

```
fclose(archivo);
```

```
Escribe texto (escribe 'fin' para terminar):  
holaaaaa, xd  
fin
```

```
Datos guardados correctamente
```

```
.....  
(program exited with code: 0)  
Press return to continue
```



MEJORA

- Validar correctamente los datos ingresados.
- Corregir mensajes repetidos.
- Leer los datos posteriormente usando `fread()`.
- Evitar funciones innecesarias como `__fpurge`.

```
if(archivo == NULL)
{
    perror("Error al crear archivo");
}
else
{
    fwrite(&a, sizeof(int), 1, archivo);
    fwrite(&b, sizeof(float), 1, archivo);
    fwrite(&c, sizeof(char), 1, archivo);
}
```

```
Escribe texto (escribe 'fin' para terminar):
holaaaaa, xd
fin
```

```
Datos guardados correctamente
```

```
-----
(program exited with code: 0)
Press return to continue
```



CONCLUSIONES



Al realizar este portafolio logré reforzar mis conocimientos relacionados con el lenguaje de programación C mediante la elaboración de diferentes programas que tienen relación con muchos de los temas básicos de los que la programación de ordenadores se ocupa tales como estructuras de control, funciones, arreglos, apuntadores y manejo de archivos.

Con cada práctica aprendí a poner en práctica el desarrollo de algoritmos más ordenados, las instrucciones de entrada y salida, operadores y los distintos tipos de datos, pero además comprendí la importancia de atender a la manera de argumentar el código, utilizando una función para no hacer código innecesariamente innecesario y fácil de solucionar.

Igualmente perfeccioné mis habilidades en el manejo de arreglos unidimensionales y bidimensionales, así como también de apuntadores y archivos de texto y binarios. Gracias a estas actividades logré mejorar mi lógica de programación, errar en los errores y mejorar programas para tener un resultado más claro, explícito y efectivo.

Por lo tanto y por conclusión; a partir de este portafolio llegué a una mejor y más clara base de la programación estructurada y que a su vez fue una gran base para el desarrollo de los próximos cursos y proyectos vinculados a las programación.



REFERENCIAS



- Gerónimo Castillo, G. (s. f.). Página personal y material académico. Universidad Tecnológica de la Mixteca.
<https://www.utm.mx/~gcgero/>
- Google Classroom. (s. f.). Material de clase (contenido en plataforma educativa).
<https://classroom.google.com/c/ODM1NTg5MDg5NDc2/m/ODYyMjQ4NzIxNjY3/details>
- Cairó Battistutti, O. (2005). Metodología de la programación: algoritmos, diagramas de flujo y programas (3.ª ed.). Alfaomega.
- Kernighan, B. W., & Ritchie, D. M. (1988). El lenguaje de programación C con base en el ANSI C (2.ª ed.). Prentice Hall.



ANEXOS



CODIGOS COMPLETOS

OPERADORES

1

ORIGINAL

```
#include<stdio.h>
int main()

{
    int a, b, c, max;

    max=1.0;
    printf("\n\t\t\tDados 3 números mostrar cual es el minimo \n");

    printf("\n\t\t\tIngresa el primer numero:\t");
    scanf("%d", &a);

    printf("\n\t\t\tIngresa el segundo:\t");
    scanf("%d", &b);

    printf("\n\t\t\tIngresa el tercer numero:\t");
    scanf("%d", &c);

    if(a>max)
        (max=a);
    if(b>max)
        (max=b);
    if(c>max)
        (max=c);

    printf("\n\t\t\tEl maximo es: %d", max);

    return 0;
}
```




```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c, max;
```

```
    printf("\n\tDatos 3 numeros mostrar cual es el maximo\n");
```

```
    printf("\nIngrese el primer numero: ");
```

```
    scanf("%d", &a);
```

```
    printf("Ingrese el segundo numero: ");
```

```
    scanf("%d", &b);
```

```
    printf("Ingrese el tercer numero: ");
```

```
    scanf("%d", &c);
```

```
    max = a;
```

```
    if(b > max)
```

```
    {
```

```
        max = b;
```

```
    }
```

```
    if(c > max)
```

```
    {
```

```
        max = c;
```

```
    }
```

```
    printf("\nEl maximo es: %d\n", max);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int n, i;
    unsigned long long factorial = 1;

    // Verifica que el usuario ingrese un número
    if(argc != 2)
    {
        printf("Uso: %s <numero>\n", argv[0]);
        return EXIT_FAILURE;
    }

    // Convierte el argumento a entero
    n = atoi(argv[1]);

    // Verifica si el número es negativo
    if(n < 0)
    {
        printf("Error: no existe factorial de numeros negativos.\n");
        return EXIT_FAILURE;
    }

    // Calcula el factorial
    for(i = 1; i <= n; i++)
    {
        factorial *= i;
    }

    // Muestra el resultado
    printf("El factorial de %d es %llu\n", n, factorial);

    return EXIT_SUCCESS;
}
```

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int n, r = 1;
    int i;

    if(argc != 2)
    {
        printf("Debes ingresar un numero\n");
        return EXIT_FAILURE;
    }

    n = atoi(argv[1]);

    if(n < 0)
    {
        printf("No existe factorial negativo\n");
        return EXIT_FAILURE;
    }

    for(i = 1; i <= n; i++)
    {
        r = r * i;
    }

    printf("\nEl factorial de %d es %d\n", n, r);

    return EXIT_SUCCESS;
}
```




```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

    char x;
    int volado;

    srand(time(NULL));

    printf("\n\t\t--- JUEGO DEL VOLADO ---\n\n");

    printf("Elige una opcion:\n");
    printf("s) sol\n");
    printf("a) aguilas\n");
    printf("Opcion: ");

    scanf(" %c", &x);

    if (x != 's' && x != 'a') {
        printf("\nOpcion invalida\n");
        return 1;
    }

    volado = rand() % 2;

    if (volado == 0) {
        printf("\nCayo: sol\n");
    } else {
        printf("\nCayo: aguilas\n");
    }

    if ((volado == 0 && x == 's') ||
        (volado == 1 && x == 'a')) {

        printf("FELICIDADES... le atinaste\n");

    } else {

        printf("Que mal, no le atinaste\n");
    }

    return 0;
}
```



```
#include <stdio.h>
```

```
int main() {  
    int a, b, c;
```

```
    printf("\t\t---PROGRAMA QUE DADO 3 NUMEROS LOS MUESTRA EN  
    PANTALLA DE FORMA DESCENDENTE---");
```

```
    printf("\n\n\t\t\tHola soy Carolina Q. Salinas P. del 217-A. ing Fisica  
    Aplicada\t");
```

```
    printf("\n\nIntroduce el primer numero: ");  
    scanf("%d", &a);  
    printf("\nIntroduce el segundo numero: ");  
    scanf("%d", &b);  
    printf("\nIntroduce el tercer numero: ");  
    scanf("%d", &c);
```

```
    printf("\n\t\t\t Los Numeros en orden descendente son: ");
```

```
    if (a >= b && a >= c) {  
        if (b >= c)  
            printf("%d, %d, %d\n", a, b, c);  
        else  
            printf("%d, %d, %d\n", a, b, c);  
    }  
    else if (b >= a && b >= c) {  
        if (a >= c)  
            printf("%d, %d, %d\n", b, a, c);  
        else  
            printf("%d, %d, %d\n", b, c, a);  
    }  
    else {  
        if (a >= b)  
            printf("%d, %d, %d\n", c, a, b);  
        else  
            printf("%d, %d, %d\n", c, b, a);  
    }
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main() {
```

```
    int a, b, c, temp;
```

```
    printf("\n--- ORDENAR 3 NUMEROS DE FORMA DESCENDENTE ---\n");
```

```
    printf("\nIntroduce el primer numero: ");
```

```
    scanf("%d", &a);
```

```
    printf("Introduce el segundo numero: ");
```

```
    scanf("%d", &b);
```

```
    printf("Introduce el tercer numero: ");
```

```
    scanf("%d", &c);
```

```
    if (a < b) {
```

```
        temp = a;
```

```
        a = b;
```

```
        b = temp;
```

```
    }
```

```
    if (a < c) {
```

```
        temp = a;
```

```
        a = c;
```

```
        c = temp;
```

```
    }
```

```
    if (b < c) {
```

```
        temp = b;
```

```
        b = c;
```

```
        c = temp;
```

```
    }
```

```
    printf("\nNumeros en orden descendente: ");
```

```
    printf("%d, %d, %d\n", a, b, c);
```

```
    return 0;
```

```
}
```

```
#include<stdio.h>
int main()
{
    int a,b,c,min;
    min=999999999;

    printf("\t\t\tDatos 3 números mostrar cual es el minimo \n");

    printf("\n\t\t\tIngrese el primer numero:\t");
    scanf("%d", &a);

    printf("\n\t\t\tIngrese el segundo numero:\t");
    scanf("%d", &b);

    printf("\n\t\t\tIngrese el tercer numero:\t");
    scanf("%d", &c);

    if(a<min)
    (min=a);
    if(b<min)
    (min=b);
    if(c<min)
    (min=c);

    printf("\n\t\t\tEl minimo es: %d", min);

    return 0;
}
```




```
#include <stdio.h>
```

```
int main() {
```

```
    int a, b, c, min;
```

```
    printf("\n\t--- ENCONTRAR EL NUMERO MINIMO ---\n");
```

```
    printf("\nIngrese el primer numero: ");  
    scanf("%d", &a);
```

```
    printf("Ingrese el segundo numero: ");  
    scanf("%d", &b);
```

```
    printf("Ingrese el tercer numero: ");  
    scanf("%d", &c);
```

```
    min = a;
```

```
    if (b < min)  
        min = b;
```

```
    if (c < min)  
        min = c;
```

```
    printf("\nEl numero minimo es: %d\n", min);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    char elc;
    float cantidad, resultado;
```

```
    printf(" \n\t Hola aqui te mostraré la Conversion básica de Unidades de
Masa\n\n");
```

```
    printf(" \n\t\t\t (recuerda escribir en mayusculas)\n\n");
```

```
    printf("\t\t\t Opción A : GRAMO (G) a KILOGRAMO (KG)\n\n");
```

```
    printf("\t\t\t Opción B : KILOGRAMO(KG) a ONZA (OZ)\n\n");
```

```
    printf("\t\t\t Opción C : ONZA (OZ) a LIBRA (LB)\n\n");
```

```
    printf("\n\t\t Selecciona una opcion: ");
```

```
    scanf(" %c", &elc);
```

```
    printf("\n\t Ingresa la cantidad a convertir: ");
```

```
    scanf("%f", &cantidad);
```

```
    switch(elc) {
```

```
        case 'A':
```

```
            resultado = cantidad / 1000;
```

```
            printf("\n\t %.2f Gramos son %.4f Kilogramos\n", cantidad, resultado);
```

```
            break;
```

```
        case 'B':
```

```
            resultado = cantidad * 35.274;
```

```
            printf("\n\t %.2f Kilogramos son %.2f Onzas\n", cantidad, resultado);
```

```
            break;
```

```
        case 'C':
```

```
            resultado = cantidad / 16;
```

```
            printf("\n\t %.2f Onzas son %.2f Libras\n", cantidad, resultado);
```

```
            break;
```

```
        default:
```

```
            printf("\n\t Opcion no valida\n");
```

```
    }
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>

int main() {

    char elc;
    float cantidad, resultado;

    printf("\n\t--- CONVERSION DE UNIDADES DE MASA ---\n");

    printf("\nA) Gramos a Kilogramos");
    printf("\nB) Kilogramos a Onzas");
    printf("\nC) Onzas a Libras");

    printf("\n\nSelecciona una opcion: ");
    scanf(" %c", &elc);

    printf("Ingresa la cantidad: ");
    scanf("%f", &cantidad);

    if (cantidad < 0) {
        printf("\nCantidad invalida\n");
        return 1;
    }

    switch(elc) {

        case 'A':
        case 'a':
            resultado = cantidad / 1000;
            printf("\n%.2f g = %.4f kg\n", cantidad, resultado);
            break;

        case 'B':
        case 'b':
            resultado = cantidad * 35.274;
            printf("\n%.2f kg = %.2f oz\n", cantidad, resultado);
            break;

        case 'C':
        case 'c':
            resultado = cantidad / 16;
            printf("\n%.2f oz = %.2f lb\n", cantidad, resultado);
            break;

        default:
            printf("\nOpcion no valida\n");
    }

    return 0;
}
```



ESTRUCTURA DE CONTROL

INTERACTIVAS

1

```
#include<stdio.h>
#include<stdlib.h>
#include<math.h>

int main()
{
    int n;
    double s=0.0, x, termino;

    printf("\t\t\t----SUMA DE ELEMENTOS de las suceción  $a_n=1/2^n$  ----\n");

    printf("\t\t\t\t\tLimite superior de la suma:\t");

    scanf ("%lf", &x);

    if(x>=1)
    {
        for (n=1; n<=x; n++)
        {
            termino =(1.0/ pow(2,n));
            s=s+termino;
        }
        printf("\n\t\tTu sumatoria es de =%.5lf",s);
    }else{
        printf("x>=1/n");
    }

    return EXIT_SUCCESS;
}
```



```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
```

```
int main() {

    int n, x;
    double suma = 0.0, termino;

    printf("\n\t--- SUMA DE LA SUCESSION an = 1 / 2^n ---\n");

    printf("\nLimite superior de la suma: ");
    scanf("%d", &x);

    if (x >= 1) {

        for (n = 1; n <= x; n++) {

            termino = 1.0 / pow(2, n);
            suma += termino;

            printf("Termino %d = %.5lf\n", n, termino);
        }

        printf("\nLa sumatoria es = %.5lf\n", suma);

    } else {

        printf("\nError: el valor debe ser mayor o igual a 1\n");
    }

    return 0;
}
```



```
#include <stdio.h>
```

```
int main()  
{  
float k, m, gravedad=9.81;  
float m_inicial, m_final, incremento;
```

```
//Datos del rsorte
```

```
printf("\t\t---PROGRAMA PARA PRACTICAR INTERVALOS CON IF--- \t\n");
```

```
printf("\n\t°°°Hola soy Carolina Q. Salinas P. del 217-A. ing. Fisica Aplicada°°°  
\t\n");
```

```
printf("\n\t\t¡El presente programa te muestra la ley de hooke! :) \t\n");
```

```
printf("\n\n\t\tIngresa un valor de tu constante K:");  
scanf("%f", &k);
```

```
//Definir intervalo
```

```
printf("\n\n\t\tDefine el intervalo de masas a utilizar!\t\n");  
printf("\n\n\t\tDame tu masa mínima:\t");  
scanf("%f", &m_inicial);  
printf("\n\n\t\tDame tu masa máxima:\t");  
scanf("%f", &m_final);  
printf("\n\n\t\tIncremento de masa:\t");  
scanf("%f", &incremento);
```



```
// resultados.
```

```
printf ("\n\t\t\tLos resultados obtenidos fueron:");
```

```
for (m=m_inicial; m<m_final; m+=incremento)
```

```
{
```

```
float fuerza=m*gravedad;
```

```
float x= fuerza/k;
```

```
printf("\n\n\nPara una masa de %.2f la fuerza es de %.2f y el estiramiento  
es de %.2f", m, fuerza,x);
```

```
}
```

```
printf("\n\n\t\t\tGRACIASSSS. Eso fue todo!");
```

```
return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main() {
```

```
    float k, m, gravedad = 9.81;  
    float m_inicial, m_final, incremento;  
    float fuerza, x;
```

```
    printf("\n\t--- LEY DE HOOKE ---\n");
```

```
    printf("\nIngresa el valor de la constante k: ");  
    scanf("%f", &k);
```

```
    printf("\nMasa minima: ");  
    scanf("%f", &m_inicial);
```

```
    printf("Masa maxima: ");  
    scanf("%f", &m_final);
```

```
    printf("Incremento de masa: ");  
    scanf("%f", &incremento);
```

```
    if (k <= 0 || incremento <= 0) {
```

```
        printf("\nDatos invalidos\n");
```

```
    } else {
```

```
        printf("\nResultados:\n");
```

```
        for (m = m_inicial; m <= m_final; m += incremento) {
```

```
            fuerza = m * gravedad;  
            x = fuerza / k;
```

```
            printf("\nMasa: %.2f | Fuerza: %.2f | Estiramiento: %.2f",  
                  m, fuerza, x);
```

```
        }
```

```
    }
```

```
    printf("\n\nGracias. Fin del programa.\n");
```

```
    return 0;
```

```
}
```




FUNCIONES

1

```
#include <stdio.h>
#include <stdlib.h>
#include <stdio_ext.h>
#include <string.h>
```

```
typedef struct {
    int folio;
    char nombre[100];
    char sexo;
    int edad;
    char tds[5];
    int presion[2];
    float temp;
    float talla;
} DATOS;
```

```
void altas(DATOS p[], int x);
void mostrar(DATOS p[], int x);
```

```
int main()
{
    DATOS paciente[50];
    int opcion, contador=0;
```

```
    do{
        system ("clear");
        printf("=====");
        printf("\n\tBINVENIDE aaaa \t ☺ CLINICA CLINATOLOGICA ☺ \n");
        printf("\t\t\t\t☺");
        printf("\n=====");
```

```
        printf("\n\n\t\t(1) Dar de alta a un nuevo paciente\n");
        printf("\n\n\t\t(2) Buscar a algún paciente\n");
        printf("\n\n\t\t¿Qué deaceas hacer?: ");
        scanf("%d", &opcion);
        __fpurge(stdin);
```



```
switch(opcion)
{
    case 1:
        altas(paciente, contador);
        contador++;
        break;
    case 2:
        mostrar(paciente, contador);
        break;
}

} while(opcion != 3);

return 0;
}

void altas(DATOS p[], int x) {
    system ("clear");

    printf("\n\t\t\t\t\tDatos del paciente 🧐\n");

    p[x].folio = x + 5;
    printf("\n\nIngresa el folio del paciente :");
    scanf("%d", &p [x].folio);
    __fpurge(stdin);

    printf("\nNombre Completo: ");
    fgets(p[x].nombre, 100, stdin);

    printf("\nSexo: ");
    scanf("%c", &p[x].sexo);
    __fpurge(stdin);

    printf("\nEdad: ");
    scanf("%d", &p[x].edad);
    __fpurge(stdin);

    printf("\nTipo de sangre: ");
    scanf("%s", p[x].tds);
    __fpurge(stdin);
```



```

printf("\nPresion arterial\n");
printf("Sistolica: ");
scanf("%d", &p[x].presion[0]);

printf("Diastolica: ");
scanf("%d", &p[x].presion[1]);

printf("\nTemperatura: ");
scanf("%f", &p[x].temp);

printf("\nTalla: ");
scanf("%f", &p[x].talla);
__fpurge(stdin);

printf("\n\t\t\tSe guardaron correctamente los datos\n");

system("clear");

__fpurge(stdin);
system("clear");
}

void mostrar(DATOS p[], int x)
{
    int folio_buscado,i,encontrado=0;
    system ("clear");

printf("\n=====
=====");
    printf("\n\t\tEXPEDIENTE \t CLINICA CLINATOLOGICA \t\n");
    printf("\t\t\t\t\t\t\t");

printf("\n=====
=====");

```



```
printf("\n\nIngrese folio del paciente 😊\t: ");
scanf("%d", &folio_buscado);
__fpurge(stdin);

for(i=0;i<x;i++)
{
    if(p[i].folio==folio_buscado)
    {
        printf("\n--- PACIENTE ENCONTRADO ---\n");
        printf("\nFolio: %d\n", p[i].folio);
        printf("\nNombre: %s", p[i].nombre);
        printf("\nSexo: %c\n", p[i].sexo);
        printf("\nEdad: %d\n", p[i].edad);
        printf("\nTipo de sangre: %s\n", p[i].tds);
        printf("\nPresion: %d/%d\n", p[i].presion[0], p[i].presion[1]);
        printf("\nTemperatura: %.2f\n", p[i].temp);
        printf("\nTalla: %.2f\n", p[i].talla);
        encontrado = 1;
        break;
    }
}

if(!encontrado)
{
    printf("\nNo hay ningún registro con ese folio\n");
}

__fpurge(stdin);
printf("\nPresiona ENTER para continuar...");
getchar();
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct {
```

```
    int folio;
    char nombre[100];
    int edad;
```

```
} DATOS;
```

```
void menu();
void altas(DATOS p[], int x);
void mostrar(DATOS p[], int x);
```

```
int main() {
```

```
    DATOS paciente[50];
    int opcion, contador = 0;
```

```
    do {
```

```
        menu();
```

```
        printf("\nOpcion: ");
        scanf("%d", &opcion);
        getchar();
```

```
        switch(opcion)
        {
```

```
            case 1:
                altas(paciente, contador);
                contador++;
                break;
```

```
            case 2:
                mostrar(paciente, contador);
                break;
```

```
            case 3:
                printf("\nFin del programa\n");
                break;
```

```
        default:
```

```
            printf("\nOpcion invalida\n");
```

```
        }
```

```
    } while(opcion != 3);
```

```
    return 0;
```

```
}
```

```
void menu() {
```

```
    printf("\n===== CLINICA =====\n");
```

```
    printf("\n1) Registrar paciente");
    printf("\n2) Buscar paciente");
    printf("\n3) Salir\n");
```

```
}
```

```
void altas(DATOS p[], int x)
```

```
{
```

```
    printf("\nFolio: ");
    scanf("%d", &p[x].folio);
    getchar();
```

```
    printf("Nombre: ");
    fgets(p[x].nombre, 100, stdin);
```

```
    printf("Edad: ");
    scanf("%d", &p[x].edad);
```

```
}
```

```
void mostrar(DATOS p[], int x)
```

```
{
```

```
    int folio, i;
```

```
    printf("\nIngresa el folio: ");
    scanf("%d", &folio);
```

```
    for(i = 0; i < x; i++)
```

```
    {
```

```
        if(p[i].folio == folio)
```

```
        {
```

```
            printf("\nPaciente encontrado\n");
```

```
            printf("Nombre: %s", p[i].nombre);
            printf("Edad: %d\n", p[i].edad);
```

```
        }
```

```
    }
```

```
}
```

DEFINICIÓN DE FUNCIONES

```
include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <stdio_ext.h>
```

```
void estrado(void);
void cabeza(void);
void cuerpo(void);
void extremidad(void);
void imprimirJuego(char*aux, int vidas, char letra, int flag);
void imprimirMarco(void);
```

```
int main()
{
    int x, i, flag, vidas, ganar;
    char materia[10][13] = {"temas\0", {"calculo\0"}, {"mecanica\0"},
{"algebra\0"}, {"ingles\0"}, {"quimica\0"}, {"programacion\0"}, {"Diseño\0"},
{"filosofia\0"}, {"logica\0"}};
    char aux[13];
    char letra;
```

```
srand(time(NULL));
```

```
x = rand() % 10;
```

```
for (i = 0; i < strlen(materia[x]); i++)
    aux[i] = '*';
```

```
aux[i] = '\0';
```

```
imprimirMarco();
```

```
printf("\n -Adivina la materia mas odiada por los alumnos-\n");
printf("\n\nmateria: %s\n", aux);
estrado();
```

```
vidas = 3;
ganar = 0;
```



```
while((vidas != 0) && (ganar != 1))
{
    printf("\n\n\tTienes %d vidas", vidas);

    printf("\n\n\tmateria:%s\n", aux);
    printf("\n Ingresa una letra:");
    __fpurge(stdin);
    scanf(" %c", &letra);

    flag = 0;

    for (i = 0; i < strlen(materia[x]); i++)
    {
        if (letra == materia[x][i])
        {
            aux[i] = letra;
            flag = 1;
        }
    }

    if (flag == 0)
    {
        vidas--;
        system("clear");
        cabeza();

        for (i = 0; i < strlen(materia[x]); i++)
            printf("%c", aux[i]);

        if (vidas == 1)
        {
            system("clear");
            cuerpo();
        }

        if (vidas == 0)
        {
            system("clear");
            extremidad();

            imprimirJuego(aux, vidas, letra, flag);
        }
    }
}
```



```

printf("\n\n\t----QUE BOT----\n\n\t Perdiste!!\n");
printf("\n\tLa palabra era: %s\n", materia[x]);
}
}
else
{
system("clear");
cuerpo();

for (i = 0; i < strlen(materia[x]); i++)
printf("%c", aux[i]);

if(strcmp(materia[x], aux) == 0)
{
ganar = 1;

imprimirJuego(aux, vidas, letra, flag);

printf("\n\n\t -----FELICITACIONES----- \n\n\t\t Ganaste!!!");
}
}
}

return EXIT_SUCCESS;
}

void imprimirMarco(void) {

printf("
|
|
| \n");
printf(" | JUEGO DEL AHORCADO | \n");
printf("
|
|
| \n");
}

void imprimirJuego(char *aux, int vidas, char letra, int flag) {
system("clear");
imprimirMarco();

```




```
}
```

```
if (vidas == 3) estrado();
else if (vidas == 2) cabeza();
else if (vidas == 1) cuerpo();
else extremidad();
```

```
printf("\n\tPalabra: %s\n", aux);
printf("\tVidas: %d\n", vidas);
```

```
if (flag == 0 && letra != 0)
printf("\tLa letra %c no esta en la palabra\n", letra);
```

```
printf(" _____\n");
}
```

```
void estrado(void) {
printf("\t\t\t __⌞__\n");
printf("\t\t\t | \n");
printf("\t\t\t | \n");
printf("\t\t\t | \n");
printf("\t\t\t | \n");
printf("\t\t\t ____\n");
printf("\t\t\t _____\n");
}
```

```
void cabeza(void) {
printf("\t\t Uyyyyy!! piensa bien tu siguiente movimiento\n");
printf("\t\t\t __⌞__\n");
printf("\t\t\t | \n");
printf("\t\t\t | O\n");
printf("\t\t\t | \n");
printf("\t\t\t | \n");
printf("\t\t\t ____\n");
printf("\t\t\t _____\n");
}
```




```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

```
void imprimirMarco(void);
```

```
int main() {
```

```
    int x, i, flag, vidas = 3, ganar = 0;
    char materia[5][15] = {
        "calculo",
        "mecanica",
        "algebra",
        "quimica",
        "programacion"
    };
```

```
    char aux[15];
    char letra;
```

```
    srand(time(NULL));
```

```
    x = rand() % 5;
```

```
    for(i = 0; i < strlen(materia[x]); i++)
        aux[i] = '*';
```

```
    aux[i] = '\0';
```

```
    imprimirMarco();
```

```
    while(vidas > 0 && ganar == 0)
    {
        printf("\nPalabra: %s\n", aux);
        printf("Vidas: %d\n", vidas);
```

```
        printf("Ingresa una letra: ");
        scanf(" %c", &letra);
```

```
        flag = 0;
```

```
        for(i = 0; i < strlen(materia[x]); i++)
        {
            if(letra == materia[x][i])
            {
                aux[i] = letra;
                flag = 1;
            }
        }
```

```
        if(flag == 0)
        {
            vidas--;
            printf("Letra incorrecta\n");
        }
```

```
        if(strcmp(materia[x], aux) == 0)
        {
            ganar = 1;
            printf("\nGanaste!!!\n");
        }
    }
```

```
    if(vidas == 0)
    {
        printf("\nPerdiste\n");
        printf("La palabra era: %s\n", materia[x]);
    }
```

```
    return 0;
}
```

```
void imprimirMarco(void) {
```

```
    printf("=====\n");
    printf("    JUEGO DEL AHORCADO    \n");
    printf("=====\n");
}
```

FUNCIÓN CON PASO DE PARÁMETRO POR VALOR

```
# include <stdio.h>
# include <stdlib.h>
# include <stdio_ext.h>
double suma(double,double);
double multiplicacion(double,double);

int main ()
{
    double a,b,c;
    char op;
    printf("Dame el primer numero:\n");
    scanf("%lf",&a);
    printf("Dame el segundo numero:\n");
    scanf("%lf",&b);
    __fpurge(stdin);
    printf("Presiona s si quieres saber la suma o m si quieres saber la
    multiplicacion:\n");
    scanf("%c",&op);

    switch (op)
    {
        case 's':
            printf("\n\t Eligió ver la suma \n");
            c=suma(a,b);
            printf("\n\t La suma es: %.2lf \n",c);
            break;
        case 'm':
            printf("\n\t Eligió ver la multiplicacion \n");
            c=multiplicacion(a,b);
            printf("\n\t La multiplicacion es: %.2lf \n",c);
            break;
        default: printf("Opcion no programada");
    }
    return EXIT_SUCCESS;
}

double suma(double x,double y)
{ double r;
  r=x+y;
  return(r);
}
double multiplicacion(double x,double y)
{
    double r;
    r=x*y;
    return(r);
}
```

MEJORA



```
#include <stdio.h>
#include <stdlib.h>
#include <stdio_ext.h>
```

```
double suma(double, double);
double multiplicacion(double, double);
```

```
int main()
```

```
{
    double a, b, c;
    char op;
```

```
    printf("Dame el primer numero:\n");
    scanf("%lf", &a);
```

```
    printf("Dame el segundo numero:\n");
    scanf("%lf", &b);
```

```
    __fpurge(stdin);
```

```
    printf("\nPresiona:");
    printf("\n s -> suma");
    printf("\n m -> multiplicacion");
    printf("\nOpcion: ");
```

```
    scanf("%c", &op);
```

```
    switch(op)
```

```
    {
        case 's':
            c = suma(a, b);
            printf("\nLa suma es: %.2lf\n", c);
            break;
```

```
        case 'm':
            c = multiplicacion(a, b);
            printf("\nLa multiplicacion es: %.2lf\n", c);
            break;
```

```
        default:
            printf("\nOpcion no valida\n");
    }
```

```
    return EXIT_SUCCESS;
}
```

```
double suma(double x, double y)
{
    return x + y;
}
```

```
double multiplicacion(double x, double y)
{
    return x * y;
}
```

FUNCIÓN CON PASO DE PARÁMETRO POR REFERENCIA

```
#include <stdio.h>
#include <stdlib.h>
#include <stdio_ext.h>

// Prototipos con paso por referencia
void suma(double, double, double *);
void multiplicacion(double, double, double *);

int main()
{
    double a, b, c;
    char op;

    printf("Dame el primer numero:\n");
    scanf("%lf", &a);

    printf("Dame el segundo numero:\n");
    scanf("%lf", &b);

    __fpurge(stdin);

    printf("Presiona s si quieres la suma o m si quieres la multiplicacion:\n");
    scanf("%c", &op);

    switch(op)
    {
        case 's':
            printf("\n\t Eligio ver la suma \n");
            suma(a, b, &c); // paso por referencia
            printf("\n\t La suma es: %.2lf \n", c);
            break;

        case 'm':
            printf("\n\t Eligio ver la multiplicacion \n");
            multiplicacion(a, b, &c); // paso por referencia
            printf("\n\t La multiplicacion es: %.2lf \n", c);
            break;

        default:
            printf("Opcion no programada");
    }

    return EXIT_SUCCESS;
}

/}
```

// FUNCIONES POR REFERENCIA

```
void suma(double x, double y, double *r)
{
    *r = x + y;
}
```

```
void multiplicacion(double x, double y, double *r)
{
    *r = x * y;
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
void suma(double, double, double *);
void multiplicacion(double, double, double *);
```

```
int main()
```

```
{
    double a, b, c;
    char op;
```

```
    printf("Dame el primer numero:\n");
    scanf("%lf", &a);
```

```
    printf("Dame el segundo numero:\n");
    scanf("%lf", &b);
```

```
    printf("Elige operacion (s = suma, m = multiplicacion):\n");
    scanf(" %c", &op);
```

```
    switch(op)
```

```
    {
        case 's':
            suma(a, b, &c);
            printf("\nResultado de la suma: %.2lf\n", c);
            break;
```

```
        case 'm':
            multiplicacion(a, b, &c);
            printf("\nResultado de la multiplicacion: %.2lf\n", c);
            break;
```

```
        default:
            printf("Opcion no valida\n");
```

```
    }
```

```
    return 0;
```

```
}
```

```
void suma(double x, double y, double *r)
```

```
{
    *r = x + y;
}
```

```
void multiplicacion(double x, double y, double *r)
```

```
{
    *r = x * y;
}
```

FUNCIÓN RECURSIVA

//Binario

```
#include <stdio.h>
#include <stdlib.h>
```

```
int binario ( int n );
int main ()
{
    int n; printf("Ingresa el valor que deces conocer en binario\t:");
    scanf("%d", & n);

    printf("\n\t\t Tu numero en binario es:");
    if(n ==0){

        printf("0");
    }else{
    }
        binario(n);
        printf("\n");

    return EXIT_SUCCESS;

}

int binario (int n)
{
    if (n>0){
        binario(n/2);
        printf("%d",n %2);
    } return 0;

}
```




```
#include <stdio.h>
#include <stdlib.h>
```

```
void binario(int n);
```

```
int main() {
    int n;
```

```
    printf("Ingresa un numero decimal: ");
    scanf("%d", &n);
```

```
    if(n < 0) {
        printf("Error: ingresa un numero positivo\n");
        return 1;
    }
```

```
    printf("\nNumero en binario: ");
```

```
    if(n == 0) {
        printf("0");
    }
    else {
        binario(n);
    }
```

```
    printf("\n");
```

```
    return 0;
}
```

```
void binario(int n)
{
    if(n > 1)
        binario(n / 2);

    printf("%d", n % 2);
}
```

**ARREGLOS
UNIDIMENSIONALES**

```
# include <stdio.h>
# include <stdlib.h>

int main()
{
    int i,j,h=0,flag;
    int A[10],B[10],C[20];

    printf("Valores del conjunto C:\n");

    for (i=0;i<10;i++)
    {
        A[i]=i+1;
        B[i]=(i+1)*2;
    }
    for (i=0;i<10;i++)
    {
        C[i]=A[i];
        h++;
    }
    for (i=0;i<10;i++)
    {
        flag=0;
        for (j=0;j<10;j++)
        {
            if(B[i]==A[j])
            {
                flag=1;
                j=11;
            }//fin del if
        }//fin del for j
        if (flag==0)
        {
            C[h]=B[i];
            h++;
        }
    }//fin del for i
    for (i=0;i<h;i++)
    printf("\nC[%d]=%d ",i,C[i]);
    return EXIT_SUCCESS;
}
```

MEJORA

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int i, j;
    int h = 0;
    int repetido;

    int A[10], B[10], C[20];

    printf("Valores del conjunto C:\n");

    for(i = 0; i < 10; i++)
    {
        A[i] = i + 1;
        B[i] = (i + 1) * 2;
    }

    for(i = 0; i < 10; i++)
    {
        C[h] = A[i];
        h++;
    }

    for(i = 0; i < 10; i++)
    {
        repetido = 0;

        for(j = 0; j < h; j++)
        {
            if(B[i] == C[j])
            {
                repetido = 1;
                break;
            }
        }

        if(!repetido)
        {
            C[h] = B[i];
            h++;
        }
    }
}
```

```
        printf("\nElementos del arreglo
C:\n");

        for(i = 0; i < h; i++)
        {
            printf("C[%d] = %d\n", i, C[i]);
        }

        return EXIT_SUCCESS;
    }
}
```

// Dada una palabra indicar cuantas vocales tiene

include <stdio.h>

include <stdlib.h>

include <string.h>

int main()

{

int i,v=0;

char palabra[10];

printf("Dame la palabra: ");

scanf("%s",palabra);

for(i=0;i<strlen(palabra);i++)

{

**if((palabra[i]!='a')&&(palabra[i]!='e')&&(palabra[i]!='i')&&
(palabra[i]!='o')&&(palabra[i]!='u'))**

v++;

}

printf("\nNumero de consonantes: %d",v);

return EXIT_SUCCESS;

}



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int i;
    int vocales = 0, consonantes = 0;

    char palabra[50];

    printf("Dame una palabra: ");
    scanf("%s", palabra);

    for(i = 0; i < strlen(palabra); i++)
    {
        if(palabra[i]=='a' || palabra[i]=='e' ||
           palabra[i]=='i' || palabra[i]=='o' ||
           palabra[i]=='u')
        {
            vocales++;
        }
        else
        {
            consonantes++;
        }
    }

    printf("\nNumero de vocales: %d", vocales);
    printf("\nNumero de consonantes: %d", consonantes);

    return EXIT_SUCCESS;
}
```

ARREGLOS
BIDIMENSIONALES

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
void estrado (void);
int main ()
{
    char palabras [7] [16]= { {"jirafa\0"}, {"hipopotamo\0"}, {"tuza\0"},
    {"mapache\0"}, {"chango\0"}, {"rroncorrorrinco\0"},
    {"pingüino\0"} };
    int x;
    // semilla
    srand(time(NULL));
    // generar numero
    x=random()%7;
    printf("palabra: %s",palabras[x]);
    estrado();
    return EXIT_SUCCESS;
}
void estrado (void)
{
    printf("\n_____");
    printf("\n |   |");
    printf("\n |");   printf(" 🦴");
    printf("\n |");   printf(" 👕");
    printf("\n |");   printf(" 👖");
    printf("\n_|_____");
}

```

MEJORA

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

void estrado(void);

int main()
{
    char palabras[7][16] =
    {
        "jirafa",
        "hipopotamo",
        "tuza",
        "mapache",
        "chango",
        "rroncorrrrinco",
        "pinguino"
    };

    int x;

    srand(time(NULL));

    x = rand() % 7;

    printf("\nPalabra seleccionada: %s\n", palabras[x]);

    estrado();

    return EXIT_SUCCESS;
}

void estrado(void)
{
    printf("\n _____");
    printf("\n |   |");
    printf("\n |   🦴");
    printf("\n |   🦺");
    printf("\n |   🦶");
    printf("\n |_____");
}
```

//Dada una matriz de 3x3 encontrar su transpuesta.

```
# include <stdio.h>
# include <stdlib.h>
# include <math.h>
int main()
{
float A[3][3];

int i,j;

printf("Valores de la matriz A\n");

for (i=0;i<3;i++)
for (j=0;j<3;j++)
{
printf("A[%d][%d]= ",i,j);
scanf("%f",&A[i][j]);
}
for (j=0;j<3;j++)
for (i=0;i<3;i++)
{A[j][i]=A[i][j];
printf("A[%d][%d]= %.1f \n",i,j,A[j][i]);
}
return EXIT_SUCCESS;
}
```


MEJORA



```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    float A[3][3];
    float T[3][3];

    int i, j;

    printf("Valores de la matriz A\n");

    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("A[%d][%d] = ", i, j);
            scanf("%f", &A[i][j]);
        }
    }

    for(i = 0; i < 3; i++)
    {
        for(j = 0; j < 3; j++)
        {
            T[i][j] = A[j][i];
        }
    }
}
```

```
printf("\nMatriz transpuesta:\n");

for(i = 0; i < 3; i++)
{
    for(j = 0; j < 3; j++)
    {
        printf("%.1f\t", T[i][j]);
    }

    printf("\n");
}

return EXIT_SUCCESS;
}
```

ARREGLO COMO PARÁMATRO

```
#include <stdio.h>
#include <stdlib.h>

// Prototipo de función (arreglo como parámetro)
int sumaArreglo(int arreglo[], int tamaño);

int main()
{
    int numeros[5];
    int i, resultado;

    // Llenar el arreglo
    printf("Ingresa 5 numeros:\n");
    for(i = 0; i < 5; i++)
    {
        printf("Numero %d: ", i + 1);
        scanf("%d", &numeros[i]);
    }

    // Llamada a la función pasando el arreglo
    resultado = sumaArreglo(numeros, 5);

    printf("\nLa suma de los elementos del arreglo es: %d\n", resultado);

    return EXIT_SUCCESS;
}

// Función que recibe un arreglo como parámetro
int sumaArreglo(int arreglo[], int tamaño)
{
    int i, suma = 0;

    for(i = 0; i < tamaño; i++)
    {
        suma = suma + arreglo[i];
    }

    return suma;
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
#define TAM 5
```

```
int sumaArreglo(int arreglo[], int tamaño);
```

```
int main()
```

```
{
```

```
    int numeros[TAM];
```

```
    int resultado;
```

```
    printf("Ingresa %d numeros:\n", TAM);
```

```
    for(int i = 0; i < TAM; i++)
```

```
    {
```

```
        printf("Numero %d: ", i + 1);
```

```
        scanf("%d", &numeros[i]);
```

```
    }
```

```
    resultado = sumaArreglo(numeros, TAM);
```

```
    printf("\nLa suma de los elementos del arreglo es: %d\n", resultado);
```

```
    return 0;
```

```
}
```

```
int sumaArreglo(int arreglo[], int tamaño)
```

```
{
```

```
    int suma = 0;
```

```
    for(int i = 0; i < tamaño; i++)
```

```
    {
```

```
        suma += arreglo[i];
```

```
    }
```

```
    return suma;
```

```
}
```

ORDENAMIENTO Y BÚSQUEDAS EN ARREGLOS

```
#include <stdio.h>
#include <stdlib.h>

// Funciones
void ordenar(int arreglo[], int n);
int buscar(int arreglo[], int n, int valor);

int main()
{
    int numeros[5];
    int i, valor, pos;

    // Llenar arreglo
    printf("Ingresa 5 numeros:\n");
    for(i = 0; i < 5; i++)
    {
        scanf("%d", &numeros[i]);
    }

    // Ordenar arreglo
    ordenar(numeros, 5);

    printf("\nArreglo ordenado:\n");
    for(i = 0; i < 5; i++)
    {
        printf("%d ", numeros[i]);
    }

    // Buscar un número
    printf("\n\nQue numero deseas buscar?: ");
    scanf("%d", &valor);

    pos = buscar(numeros, 5, valor);

    if(pos == -1)
        printf("El numero no se encontro\n");
    else
        printf("El numero esta en la posicion %d\n",
pos);

    return EXIT_SUCCESS;
}
```

```
// ORDENAMIENTO (BURBUJA)
void ordenar(int arreglo[], int n)
{
    int i, j, aux;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - 1; j++)
        {
            if(arreglo[j] > arreglo[j + 1])
            {
                aux = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = aux;
            }
        }
    }
}

// BÚSQUEDA LINEAL
int buscar(int arreglo[], int n, int valor)
{
    int i;

    for(i = 0; i < n; i++)
    {
        if(arreglo[i] == valor)
        {
            return i; // posición encontrada
        }
    }

    return -1; // no encontrado
}
```

MEJORA

```
#include <stdio.h>
#include <stdlib.h>
```

```
void ordenar(int arreglo[], int n);
int buscar(int arreglo[], int n, int valor);
```

```
int main()
{
    int numeros[5];
    int valor, pos;

    printf("Ingresa 5 numeros:\n");

    for(int i = 0; i < 5; i++)
    {
        scanf("%d", &numeros[i]);
    }

    ordenar(numeros, 5);

    printf("\nArreglo ordenado:\n");

    for(int i = 0; i < 5; i++)
    {
        printf("%d ", numeros[i]);
    }

    printf("\n\nQue numero deseas buscar?: ");
    scanf("%d", &valor);

    pos = buscar(numeros, 5, valor);

    if(pos == -1)
        printf("El numero no se encontro\n");
    else
        printf("El numero %d esta en la posicion %d\n", valor, pos);

    return 0;
}
```

```
void ordenar(int arreglo[], int n)
{
    int aux;

    for(int i = 0; i < n - 1; i++)
    {
        for(int j = 0; j < n - 1 - i; j++)
        {
            if(arreglo[j] > arreglo[j + 1])
            {
                aux = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = aux;
            }
        }
    }
}

int buscar(int arreglo[], int n, int valor)
{
    for(int i = 0; i < n; i++)
    {
        if(arreglo[i] == valor)
        {
            return i;
        }
    }
    return -1;
}
```

APUNTADORES

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    char texto[20];
    char *p, *q, *a, *h, *l;
    int cont = 0;

    printf("Ingresa alguna frase: \n");
    fgets(texto, 20, stdin);

    q = (char*) malloc(20);
    h = (char*) malloc(20);
    a = q;
    l = h;
    p = texto;
```

```
while(*p != '\0')
{
    if(*p != ' ' && *p != '\n')
    {
        *q = *p;
        q++;
        cont++;
    }
    p++;
}
```

```
*q = '\0';
```

```
printf("\nTu frase sin espacios: \n%s\n",
a);
q--;
while(cont > 0)
{
    *h = *q;
    h++;
    q--;
    cont--;
}

*h = '\0';

printf("\nTu frase invertida: \n%s\n", l);

return EXIT_SUCCESS;
}
```



```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
```

```
    char texto[100];
    char *p, *q, *a, *h, *l;
    int cont = 0;
```

```
    printf("Ingresa alguna frase:\n");
    fgets(texto, 100, stdin);
```

```
    q = (char*) malloc(100);
    h = (char*) malloc(100);
```

```
    if(q == NULL || h == NULL) {
        printf("Error al reservar memoria\n");
        return 1;
    }
```

```
    a = q;
    l = h;
    p = texto;
```

```
    while(*p != '\0')
    {
        if(*p != ' ' && *p != '\n')
        {
            *q = *p;
            q++;
            cont++;
        }
        p++;
    }
```

```
    *q = '\0';
```

```
    printf("\nTu frase sin espacios:\n%s\n", a);
```

```
    q--;
```

```
    while(cont > 0)
    {
        *h = *q;
        h++;
        q--;
        cont--;
    }
```

```
    *h = '\0';
```

```
    printf("\nTu frase invertida:\n%s\n", l);
```

```
    free(a);
    free(l);
```

```
    return 0;
}
```



ARITMÉTICA DE APUNTADES

1

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    char texto[100];
    char *p, *q, *a, *h, *l;
    int cont = 0;
```

```
    printf("Ingresa una frase: ");
    fgets(texto, 100, stdin);
```

```
    q = malloc(100);
    h = malloc(100);
```

```
    a = q;
    l = h;
```

```
    p = texto;
```

```
    while(*p != '\0')
    {
        if(*p != ' ' && *p != '\n')
        {
            *q = *p;
            q++;
            cont++;
        }
        p++;
    }
```

```
    *q = '\0';
    printf("\nTu frase sin espacios: %s\n", a);
    q--;
```

```
    while(cont > 0)
    {
        *h = *q;
        h++;
        q--;
        cont--;
    }
```

```
    *h = '\0';
```

```
    printf("\nTu frase invertida: %s\n", l);
```

```
    p = a;
    q = l;
```

```
    while(*p != '\0' && *q != '\0')
    {
        if(*p != *q)
        {
            printf("\nTu frase no es palindromo\n");
            return EXIT_SUCCESS;
        }
        p++;
        q++;
    }
```

```
    printf("\nTu frase es palindromo\n");
```

```
    return EXIT_SUCCESS;
}
```




```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
```

```
int main() {
```

```
    char texto[100];
    char *p, *q, *a, *h, *l;
    int cont = 0;
```

```
    printf("Ingresa una frase: ");
    fgets(texto, 100, stdin);
```

```
    q = malloc(100);
    h = malloc(100);
```

```
    if (q == NULL || h == NULL) {
        printf("Error de memoria\n");
        return 1;
    }
```

```
    a = q;
    l = h;
```

```
    p = texto;
```

```
    while(*p != '\0')
    {
        if(*p != ' ' && *p != '\n')
        {
            *q = tolower(*p);
            q++;
            cont++;
        }
        p++;
    }
```

```
    *q = '\0';
```

```
}
```

```
    printf("\nFrase sin espacios: %s\n", a);
```

```
    q--;
```

```
    while(cont > 0)
```

```
    {
        *h = *q;
        h++;
        q--;
        cont--;
    }
```

```
    *h = '\0';
```

```
    printf("Frase invertida: %s\n", l);
```

```
    p = a;
    q = l;
```

```
    while(*p != '\0' && *q != '\0')
```

```
    {
        if(*p != *q)
```

```
        {
            printf("\nLa frase no es palindromo\n");
```

```
            free(a);
            free(l);
```

```
            return 0;
        }
```

```
        p++;
        q++;
    }
```

```
    printf("\nLa frase es palindromo\n");
```

```
    free(a);
    free(l);
```

```
    return 0;
```



```
#include <stdio.h>
#include <stdlib.h>
#include <stdio_ext.h>
#include <string.h>

int main () {
    FILE *archivo;
    char linea [255];

    //NUEVA VARIABLE

    //abrir (crear) el archivo

    archivo=fopen("historia.dat","a" ); //nombre del archivo, modo
    if(archivo==NULL)
    {
        printf("Error en la creacion del archivo\n");
        exit(0);
    }else{
        do{
            fgets(linea, 255, stdin);

            //verificar si linea trae fin

            //nueva funcion

            if(strcmp(linea,"fin\n")==0) break;
            else

            //nueva instruccion para mandar escribir a un archivo como un formato
            fprintf(archivo,"%s",linea);

        }while (strcmp(linea,"fin\n")!=0);
    }
    //cerrar
    fclose (archivo);

    return EXIT_SUCCESS;
}
```



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
int main() {
```

```
    FILE *archivo;
    char linea[255];
```

```
    archivo = fopen("historia.dat", "a");
```

```
    if(archivo == NULL) {
```

```
        printf("Error al abrir el archivo\n");
        return 1;
```

```
    }
```

```
    printf("Escribe texto (escribe 'fin' para terminar):\n");
```

```
    do {
```

```
        fgets(linea, 255, stdin);
```

```
        if(strcmp(linea, "fin\n") == 0)
            break;
```

```
        fprintf(archivo, "%s", linea);
```

```
    } while(1);
```

```
    fclose(archivo);
```

```
    printf("\nDatos guardados correctamente\n");
```

```
    return 0;
```

```
}
```

//Funciones binarios

```
#include <stdlib.h>
#include <stdio.h>
#include <stdio_ext.h>
int main()
{
    int a;
    float b;
    char c;
    FILE *archivo;
    archivo=fopen("datos.bin","wb");
    if (archivo==NULL)
    {
        perror ("Error en la creacion del rchivo");

        return EXIT_FAILURE;
    }
    else{
        printf("Numero entero:"); __fpurge(stdin);
        scanf("%d", &a);
        printf("Numero real:");
        scanf("%f", &b);__fpurge(stdin);
        printf("Numero real:");
        scanf("%c", &c);

        fwrite(&a, sizeof(a), 1, archivo);
        fwrite(&b, sizeof(b), 1, archivo);
        fwrite(&c, sizeof(c), 1, archivo);

        fclose(archivo);
    }
    return EXIT_SUCCESS;
}
```

MEJORA



```
#include <stdio.h>
#include <stdlib.h>

int main() {

    int a;
    float b;
    char c;

    FILE *archivo;
    archivo = fopen("datos.bin", "wb");

    if(archivo == NULL)
    {
        perror("Error al crear archivo");
        return 1;
    }

    printf("Numero entero: ");
    scanf("%d", &a);
    printf("Numero real: ");
    scanf("%f", &b);

    getchar();
    printf("Caracter: ");
    scanf("%c", &c);

    fwrite(&a, sizeof(int), 1, archivo);
    fwrite(&b, sizeof(float), 1, archivo);
    fwrite(&c, sizeof(char), 1, archivo);

    fclose(archivo);

    printf("\nDatos guardados correctamente\n");

    return 0;
}
```